

Paralelismo a nivel de tareas

Septiembre/2026



Contenido

- Introducción
 - ¿Qué es paralelismo a nivel de tarea?
- Grafo computacional
- Paralelismo ideal
- Planificación (scheduling) en multiprocesadores
- Análisis de algoritmos paralelos
 - Ley/Argumento de Amdahl
 - Ley de Gustafson-Barsis
- Retos y Dificultades en la computación paralela
- Computación heterogénea
- Referencias

Introducción

¿Qué es paralelismo a nivel de tarea?

Paralelismo a nivel de tarea vs. Paralelismo a nivel de datos

- **Paralelismo a nivel de tarea:** habilidad para ejecutar al mismo tiempo diferentes tareas dentro de un problema.
- **Paralelismo a nivel de datos:** habilidad para ejecutar al mismo tiempo partes de la misma tarea sobre diferentes datos.

Analogía:

Recolección de granos de café

Recolector: procesador (hardware)

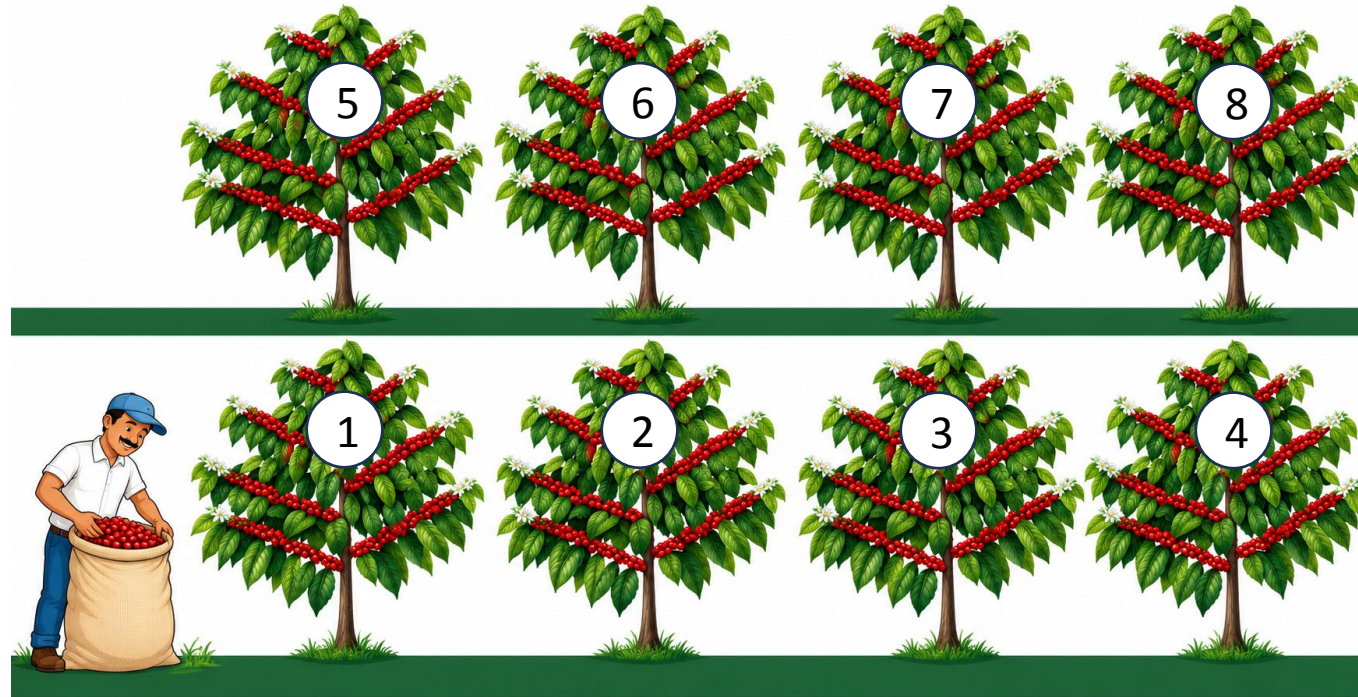
Arbusto de café: tarea

Granos de café: datos



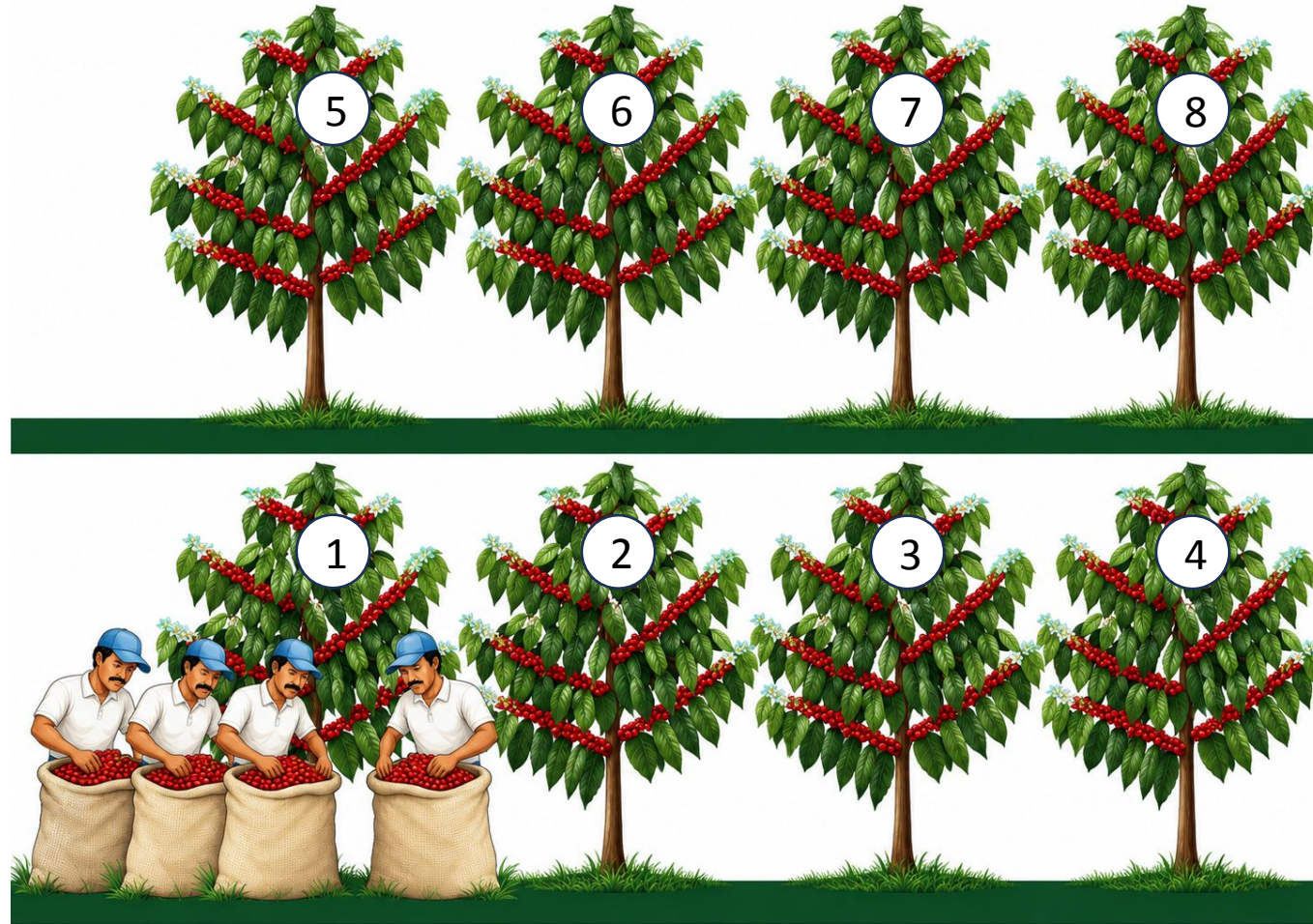
Enfoque secuencial

- Un único recolector recoge todos los granos de cada arbusto.



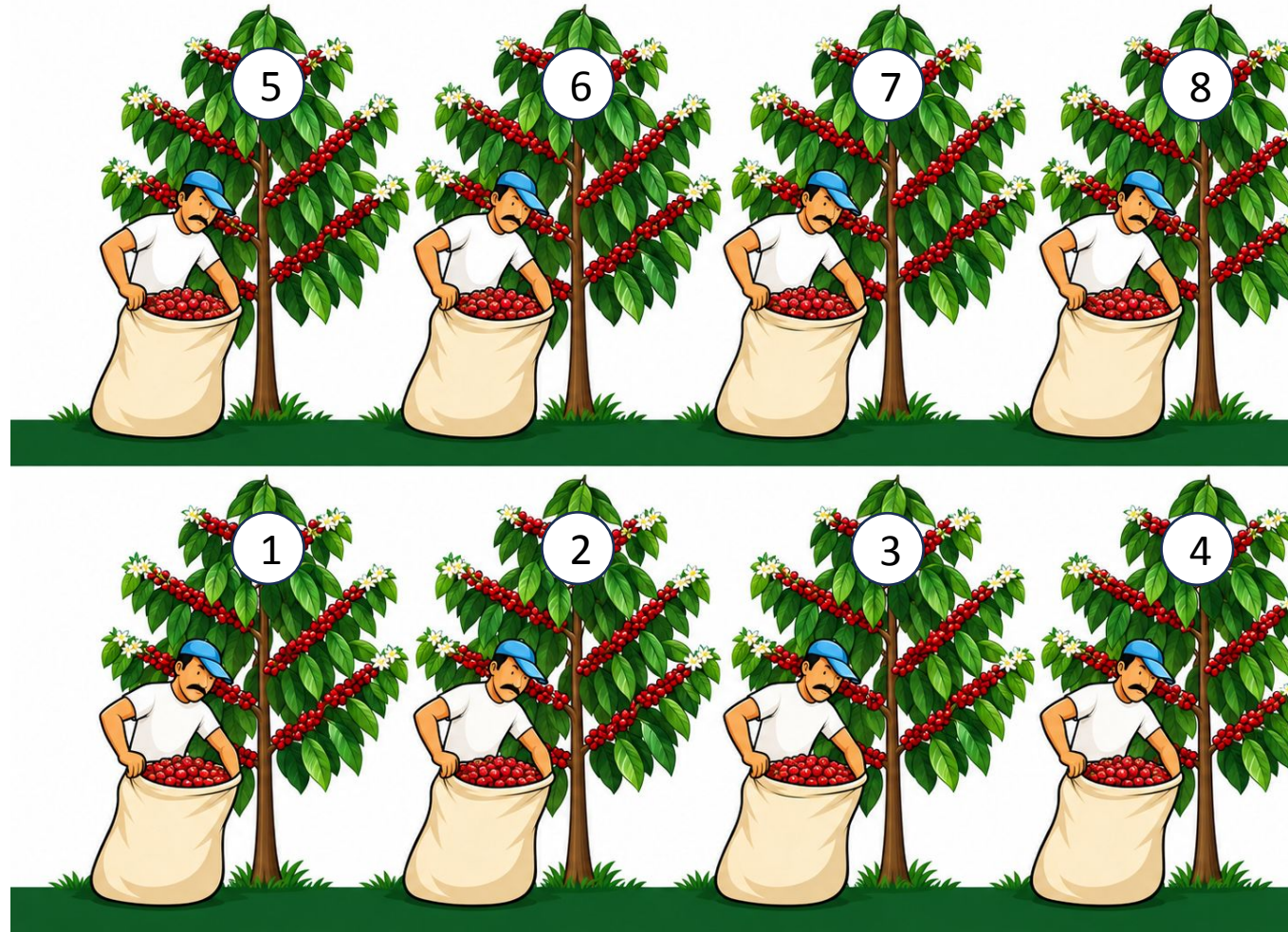
Paralelismo a nivel de datos

- Varios recolectores trabajan en el mismo arbusto hasta que terminen con los granos maduros [datos], luego pasan al siguiente arbusto.



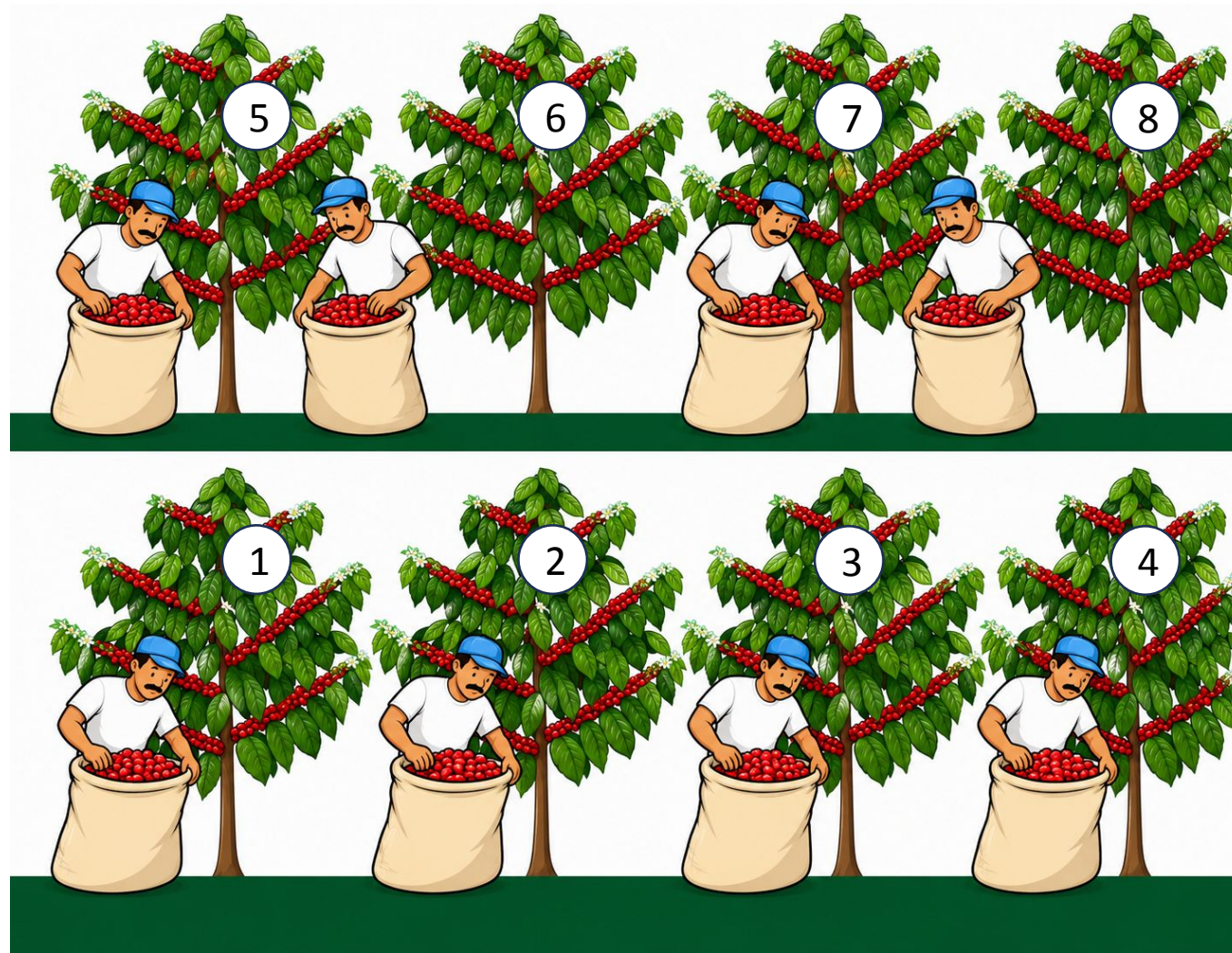
Paralelismo a nivel de tareas

- Varios recolectores, cada uno trabaja en un arbusto hasta que termine con los granos maduros de la “tarea” asignada.



Paralelismo en la vida real

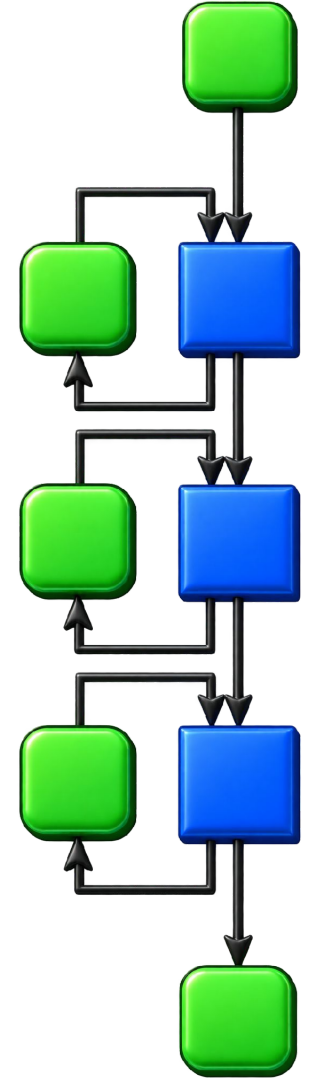
- Varios recolectores, con técnicas de recolección diversas (algunos con su arbusto “propio” y otros comparten el arbusto).



Pipelining

Es la misma línea de ensamblaje

En ocasiones lo traducen como “Tubería” o “Canalización”



Bucket Brigade



Bucket Brigade

Pipeline es diferente a Bucket Brigade (la segunda es más dinámica)

Contenido

- Introducción
 - ¿Qué es paralelismo a nivel de tarea?
- Grafo computacional
- Paralelismo ideal
- Planificación (scheduling) en multiprocesadores
- Análisis de algoritmos paralelos
 - Ley/Argumento de Amdahl
 - Ley de Gustafson-Barsis
- Retos y Dificultades en la computación paralela
- Computación heterogénea
- Referencias

Grafo Computacional

Création et réalisation de tâches

- Dîner préparé par deux personnes
 - Conception du menu
 - Les achats au supermarché
 - Préparation
 - Servir le dîner



Creación y terminación de tareas

- Cena elaborada por dos personas
 1. Diseño del menú
 2. Compra en el supermercado
 3. Preparación de la cena
 4. Servir la cena



Création et réalisation de tâches

• Conception du menu

- Apéritif: blanc de vermouth
- Entrée: crème de poulet
- Plat principal
 - Médaillons de poulet façon cordon bleu
 - Purée de pomme de terre
 - Salade César
- Sauce aux champignons
- Dessert: trois lait gélifié
- Vin blanc



Creación y terminación de tareas

•Diseño del menú

- Aperitivo: vermut blanco
- Entrada: Crema de pollo (caldo base, roux)
- Plato principal
 - Medallones de pollo a la cordon bleu
 - Salsa de champiñones
 - Puré de papa
 - Ensalada César
- Postre: Tres leches con gelatina
- Vino blanco



Se puede hacer entre varias personas al mismo tiempo

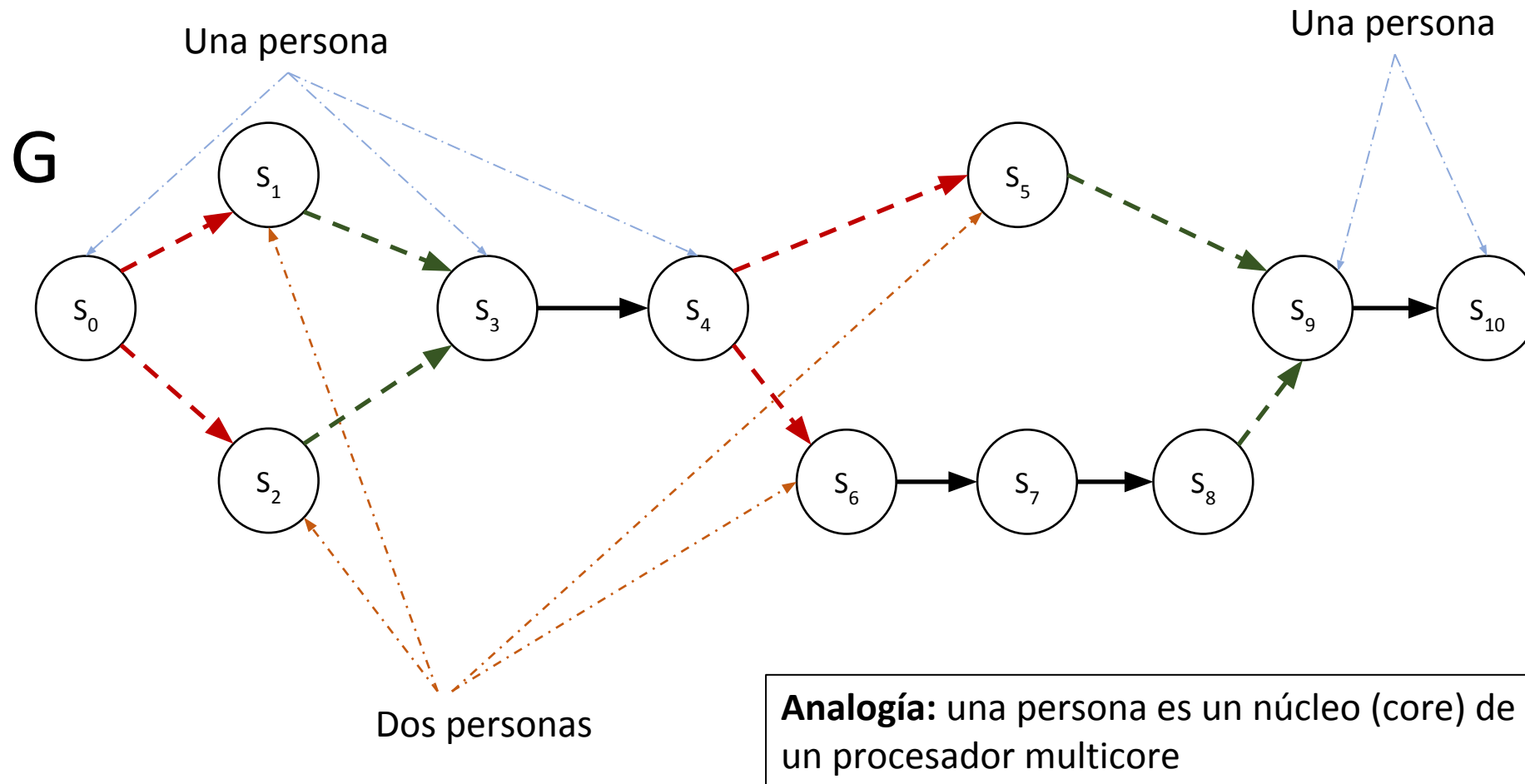
Lista de actividades

- S_0 = Diseñar el menú
- S_1 = Compra ingredientes aperitivo, vino, entrada y medallones
- S_2 = Compra ingredientes puré, ensalada, postre
- S_3 = Arreglar ingredientes
- S_4 = Organizar cocina
- S_5 = Preparar medallones
- S_6 = Preparar ensalada
- S_7 = Preparar puré
- S_8 = Preparar crema de pollo
- S_9 = Organizar el comedor
- S_{10} = Servir la cena

Nos vamos a saltar la preparación del postre y la salsa de champiñones para simplificar el ejemplo.



Grafo representando las actividades de la cena

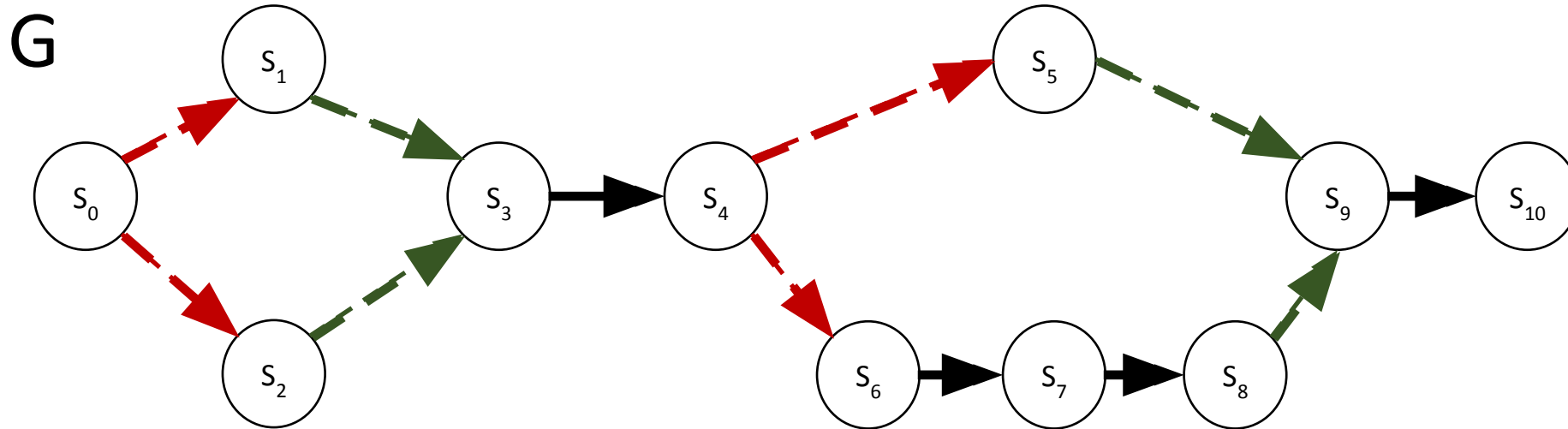


Grafo computacional

Un **grafo computacional (CG)** modela la ejecución de un programa en paralelo.

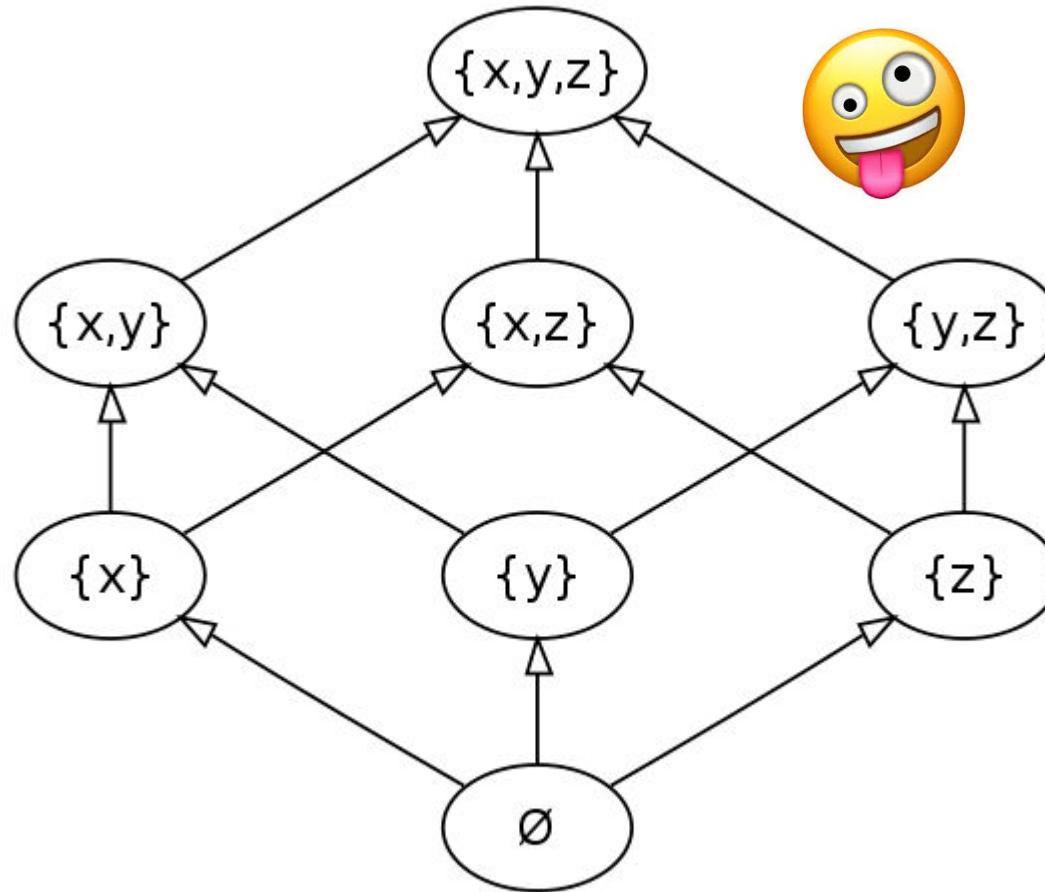
Consta de:

- Un conjunto de **vértices**
- Un conjunto de **arcos** (representan las reglas de ordenamiento)



- **Arcos de continuación** (*continue*): representan pasos secuenciales dentro de las tareas
- **Arcos de bifurcación** (*fork*): conecta la operación de bifurcación con el primer paso de la tarea hija
- **Arcos de encuentro** (*join*): conecta el último paso de la tarea hija a todas las operaciones de encuentro de la tarea padre.

Conjunto ordenado parcialmente



TAREA: ¿Qué es un POSET (Partially Ordered Set)?

Contenido

- Introducción
 - ¿Qué es paralelismo a nivel de tarea?
- Grafo computacional
- Paralelismo ideal
- Planificación (scheduling) en multiprocesadores
- Análisis de algoritmos paralelos
 - Ley/Argumento de Amdahl
 - Ley de Gustafson-Barsis
- Retos y Dificultades en la computación paralela
- Computación heterogénea
- Referencias

Paralelismo ideal

Límite superior del factor de mejora en la rapidez (*speedup*) que puede ser obtenido a partir de la ejecución paralela de los nodos en el grafo computacional G

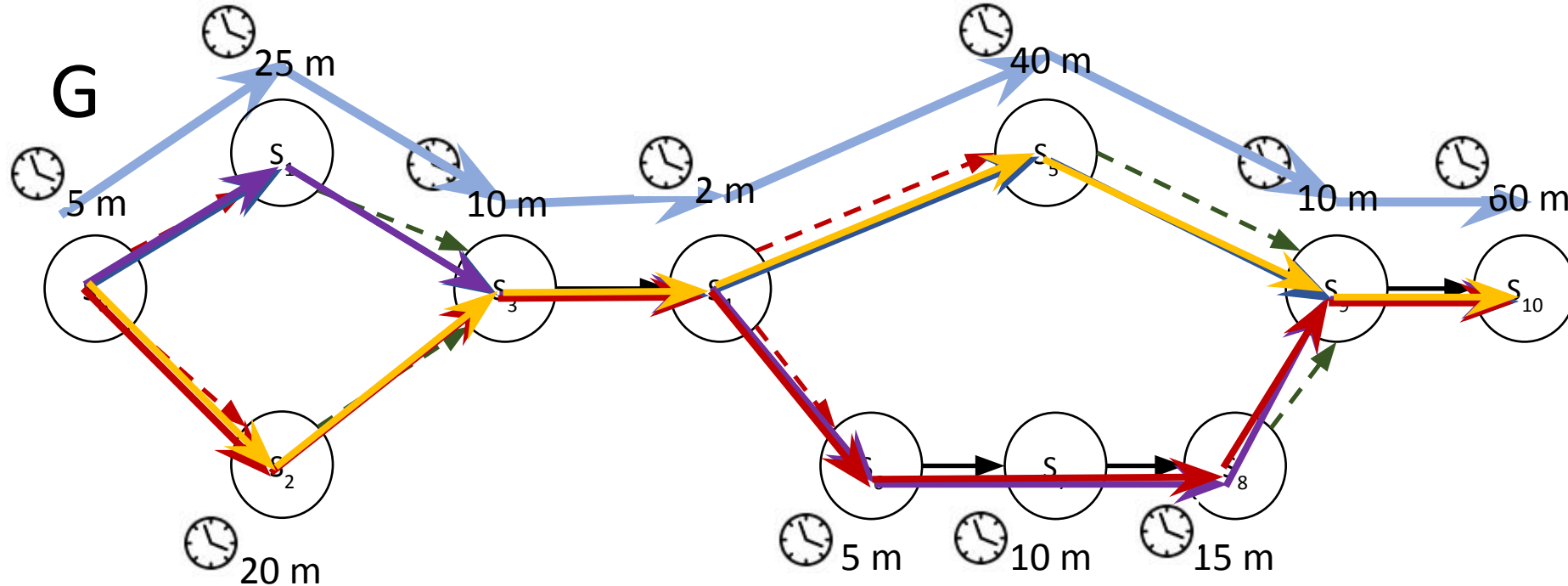
- Tiempos de ejecución para las actividades de la cena [en minutos]

- S_0 = Diseñar el menú [5 min]
- S_1 = Compra ingredientes aperitivo, entrada, medallones [25 min]
- S_2 = Compra ingredientes puré, ensalada, postre [20 min]
- S_3 = Arreglar ingredientes [10 min]
- S_4 = Organizar cocina [2 min]
- S_5 = Preparar medallones [40 min]
- S_6 = Preparar ensalada [5 min]
- S_7 = Preparar puré [10 min]
- S_8 = Preparar crema de pollo [15 min]
- S_9 = Organizar el comedor [10 min]
- S_{10} = Servir la cena [60 min]

OJO: la preparación del postre y la salsa de champiñones no están.



Paralelismo ideal



WORK (G)

Parth1 =

Parth2 =

Parth3 =

Parth4 =

Se define WORK(G) (trabajo) como la suma de los tiempos de ejecución de TODOS los nodos en el Grafo Computacional G.

Se define SPAN(G) como la longitud del trayecto más largo en G, cuando se suman los tiempos de ejecución de todos los nodos que pertenecen al trayecto.

También se llama critical path length - CPL

SPAN(G)

Paralelismo ideal

- El paralelismo ideal de un grafo de computación G se define como la razón

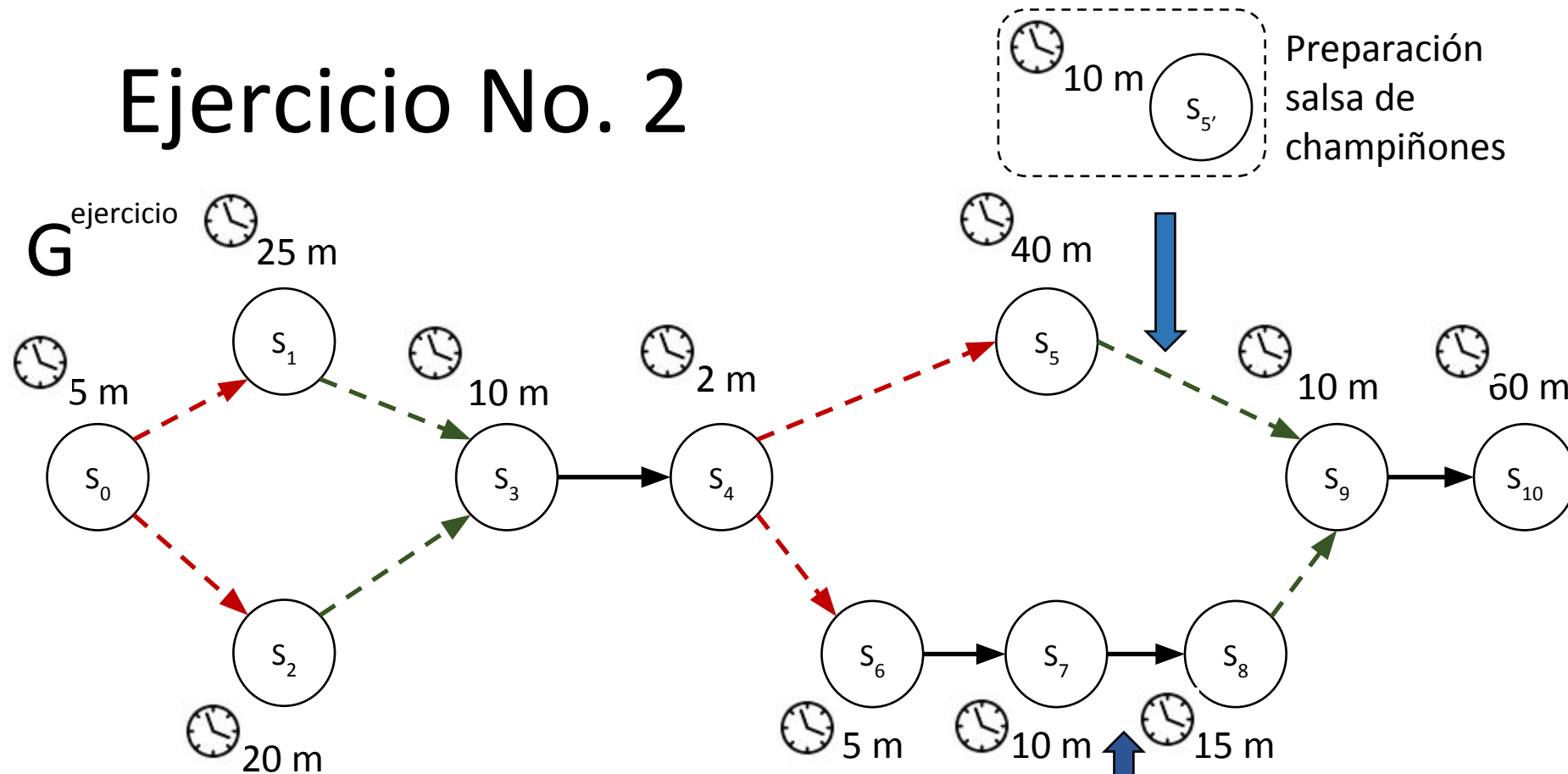
$$\text{WORK}(G)/\text{SPAN}(G)$$

- El paralelismo ideal es el límite superior del factor de mejora en la rapidez (speedup) que puede ser obtenido a partir de la ejecución paralela de los nodos en el grafo computacional G .

$$\text{SPEEDUP} \leq \text{WORK}(G)/\text{SPAN}(G)$$

- El paralelismo ideal es solo una función teórica del programa paralelo y NO depende del paralelismo real disponible en el computador físico.
- Para el ejemplo de las actividades de la cena el paralelismo ideal es de $202/152 = 1,33$

Ejercicio No. 2



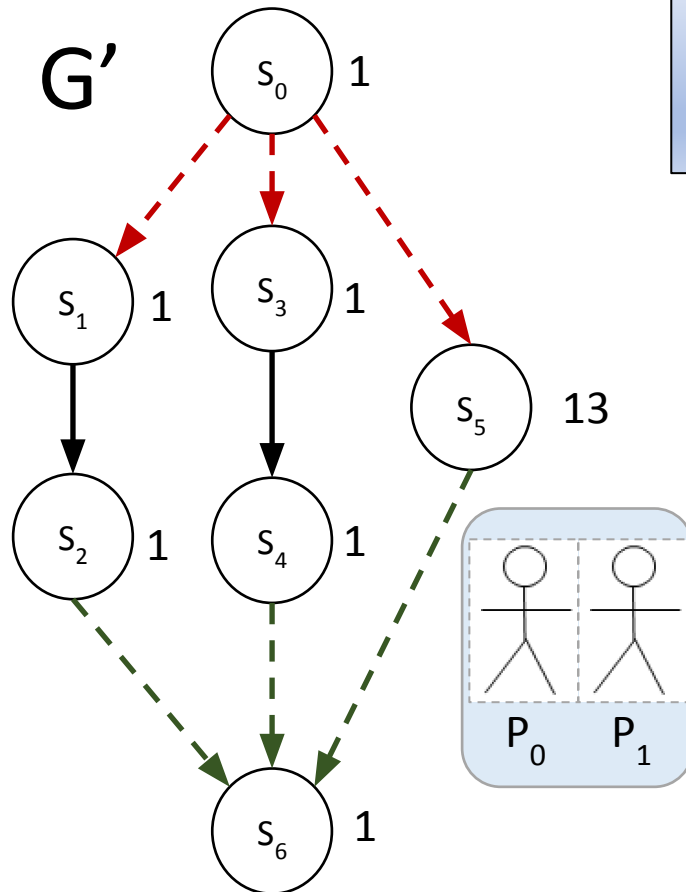
Al grafo computacional mostrado ($G^{\text{ejercicio}}$) adiciónale las dos actividades indicadas en el sitio que señalan las flechas azules en el diagrama. Después calcule el $\text{WORK}(G^{\text{ejercicio}})$, el $\text{SPAN}(G^{\text{ejercicio}})$ y el paralelismo ideal correspondiente.

Contenido

- Introducción
 - ¿Qué es paralelismo a nivel de tarea?
- Grafo computacional
- Paralelismo ideal
- Planificación (scheduling) en multiprocesadores
- Análisis de algoritmos paralelos
 - Ley/Argumento de Amdahl
 - Ley de Gustafson-Barsis
- Retos y Dificultades en la computación paralela
- Computación heterogénea
- Referencias

Planificación (scheduling) en multiprocesadores

Planificación (scheduling) en multiprocesadores: ejemplo 1



Si el procesamiento se realiza en un computador que tiene p procesadores, se dice que T_p es el tiempo transcurrido entre el inicio y la finalización del procesamiento.

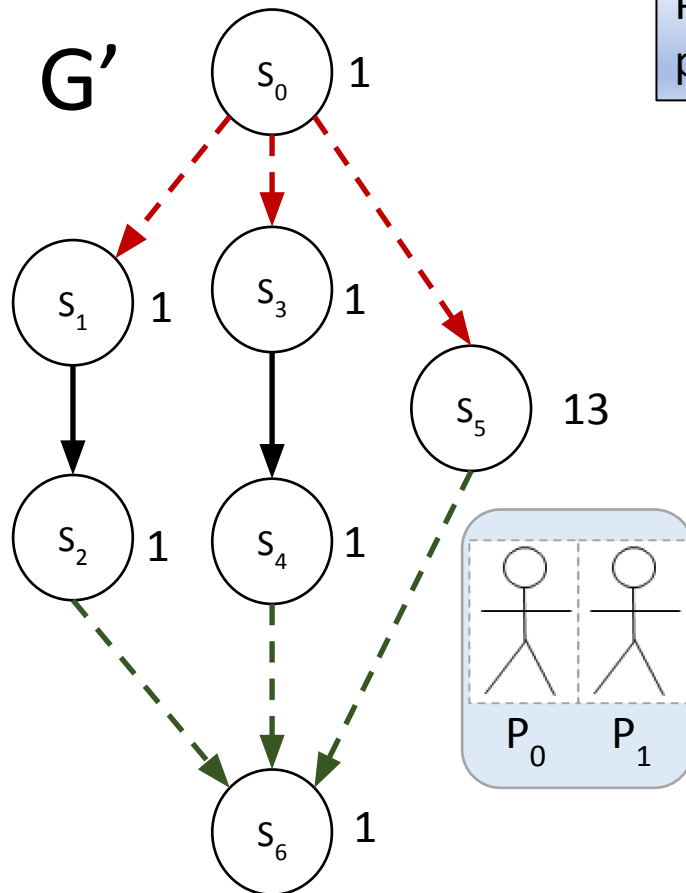
Para este ejemplo, se tiene un computador con $P = 2$ procesadores: P_0 y P_1 .

Scheduling No. 1 con $P = 2$

P_0	P_1
$s_0[1]$	Ocioso
$s_1[1]$	$s_3[1]$
$s_2[1]$	$s_4[1]$
$s_5[13]$	Ocioso [13]
$s_6[1]$	Ocioso

$T_2 = 17$

Planificación (scheduling) en multiprocesadores: ejemplo 2



Para este ejemplo, se tiene un computador con $P = 2$ procesadores: P_0 y P_1 .

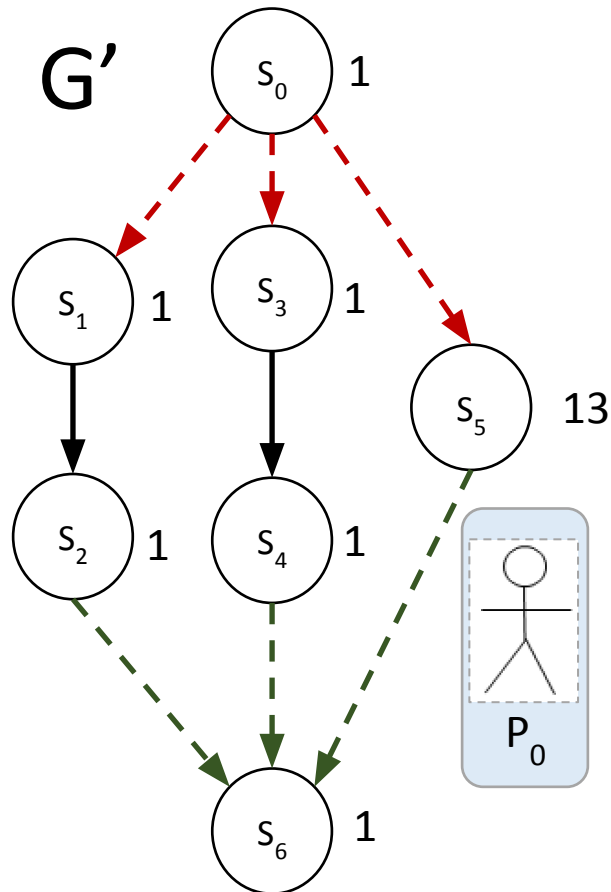
Scheduling No. 2 con $P = 2$

Priorizando S_5

P_0	P_1
$s_0[1]$	Ocioso
$s_5[13]$	$s_1[1]$
	$s_2[1]$
	$s_3[1]$
	$s_4[1]$
	Ocioso [9]
$s_6[1]$	Ocioso

$T_2 = 15$

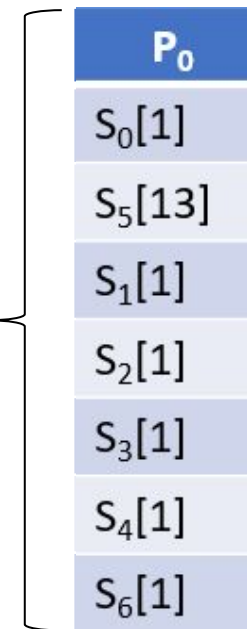
Planificación (scheduling) en multiprocesadores: ejemplo 3



Para este ejemplo, se tiene un computador con $P = 1$ procesador: P_0 .

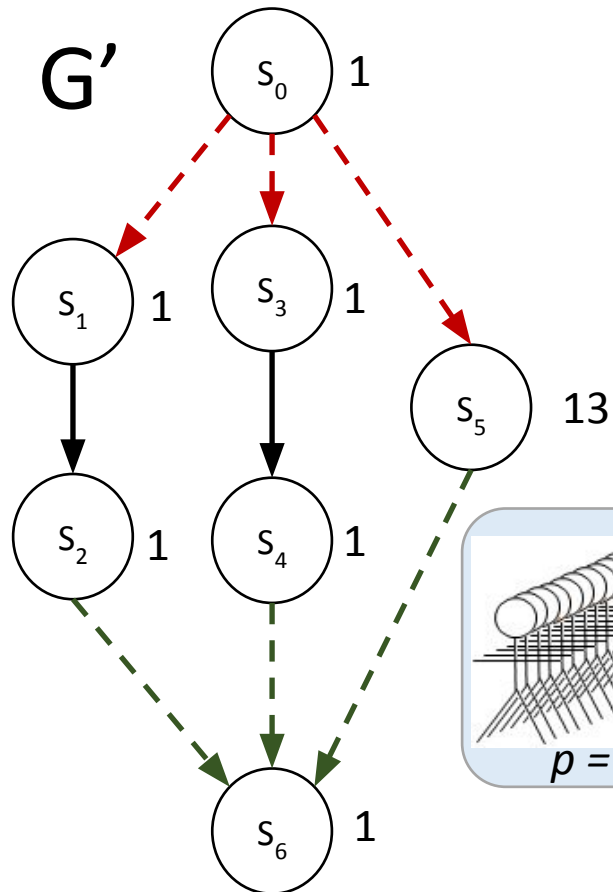
Scheduling con $P = 1$

$T_1 = 19$



$T_1 = \text{WORK}$
Para este ejemplo:
 $T_1 = 19$

Planificación (scheduling) en multiprocesadores: ejemplo 4

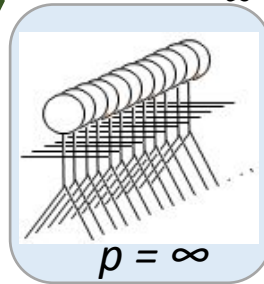


Para este ejemplo, se tiene un computador con $P = \infty$ procesadores: $P_0, P_1, P_2, P_3 \dots$

Scheduling con $P = \infty$

P_0	P_1	P_2	P_3	...
$s_0[1]$	Ocioso	Ocioso	Ocioso	Ocioso
$s_5[13]$	$s_1[1]$	$s_3[1]$	Ocioso	Ocioso
	$s_2[1]$	$s_4[1]$	Ocioso	Ocioso
	Ocioso	Ocioso	Ocioso	Ocioso
$s_6[1]$	Ocioso	Ocioso	Ocioso	Ocioso

$T_\infty = 15$



$T_\infty = \text{SPAN}$
Para este ejemplo:
 $T_\infty = 15$

Es importante resaltar que:

$$T_\infty \leq T_p \leq T_1$$

Una lección del ajedrez

- Una historia real del desarrollo de un programa paralelo de ajedrez: se probaba en una máquina de 32 procesadores, aunque estaba pensado para correr en una de 512. En un punto, los desarrolladores aplicaron una “optimización” que en 32 procesadores bajó el tiempo de una prueba de 65 a 40 segundos, pero al analizarla con **work/span** concluyeron que esa mejora no escalaría y decidieron descartarla.
- El análisis muestra por qué: la versión original tenía trabajo $T_1 = 2048s$ y span $T_\infty = 1s$, y la optimizada $T_1 = 1024s$ pero $T_\infty = 8s$.
- Con la aproximación $T_P \approx T_1/P + T_\infty$, en 32 procesadores la optimizada gana (40 s vs 65s), pero en 512 pierde (10 s vs 5s) porque el span de 8 segundos se vuelve el término dominante y “ahoga” la paralelización.

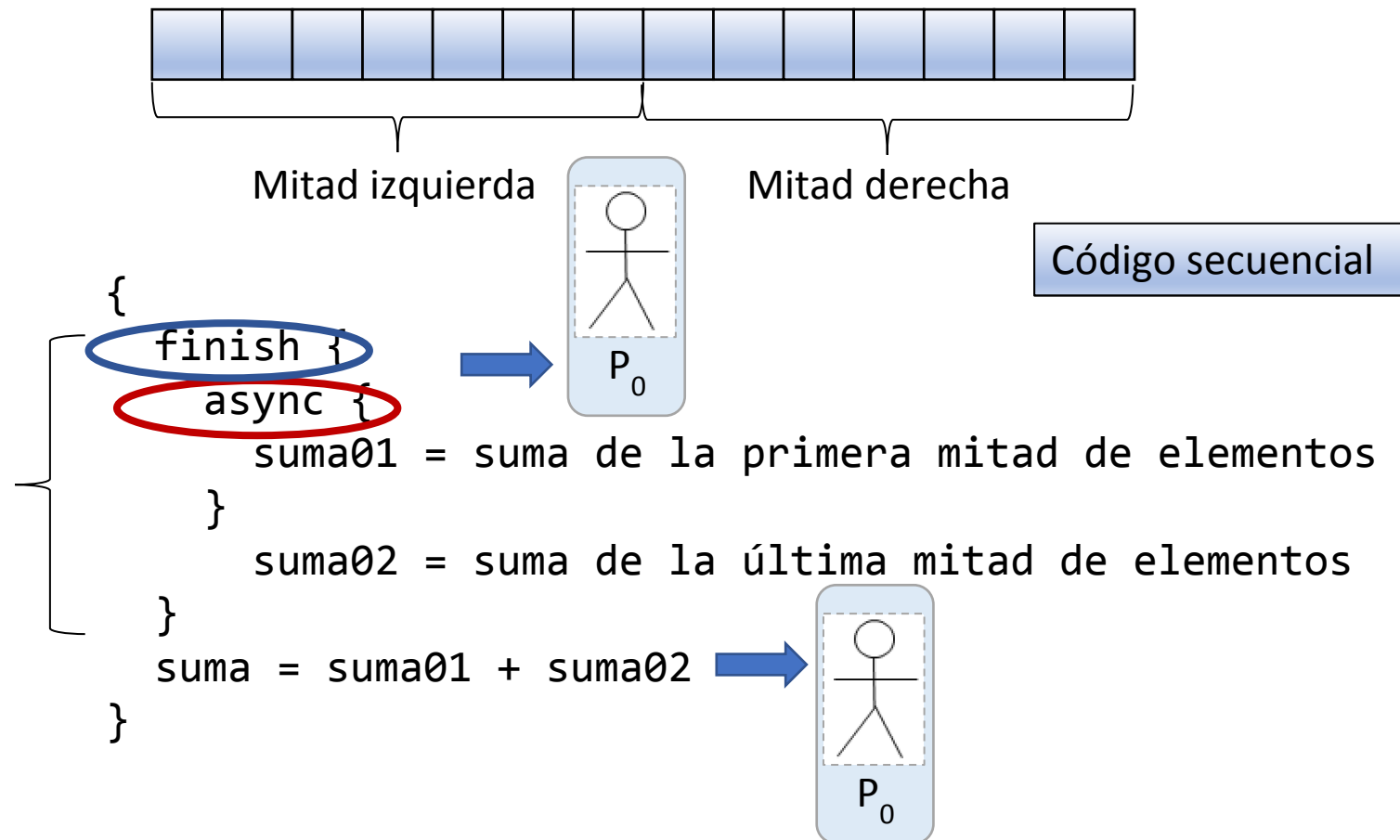


La moraleja es que medir y analizar **work** y **span** puede predecir mejor la escalabilidad que mirar solo tiempos medidos en una máquina pequeña.

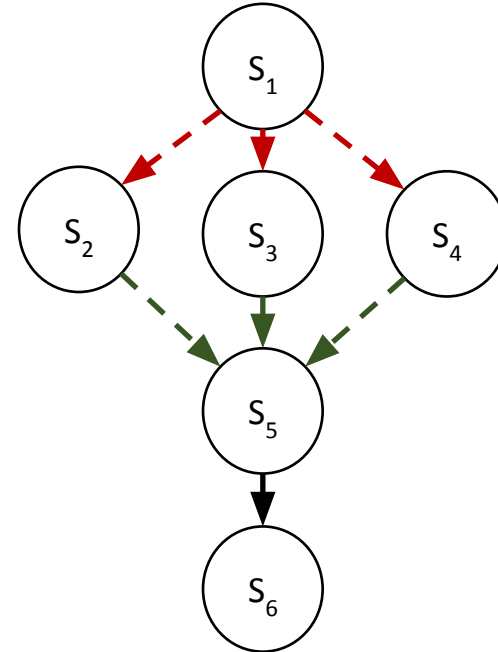
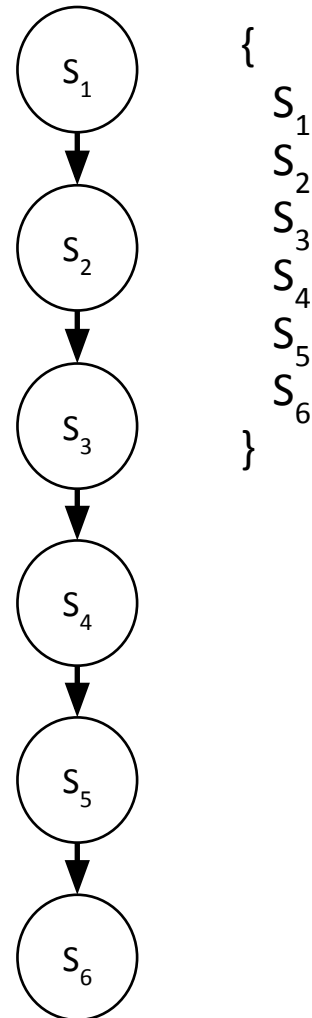
Inicio y terminación de tareas

Inicio y terminación de tareas

- Ejemplo usando `async` y `finish`: suma de las dos mitades de un arreglo



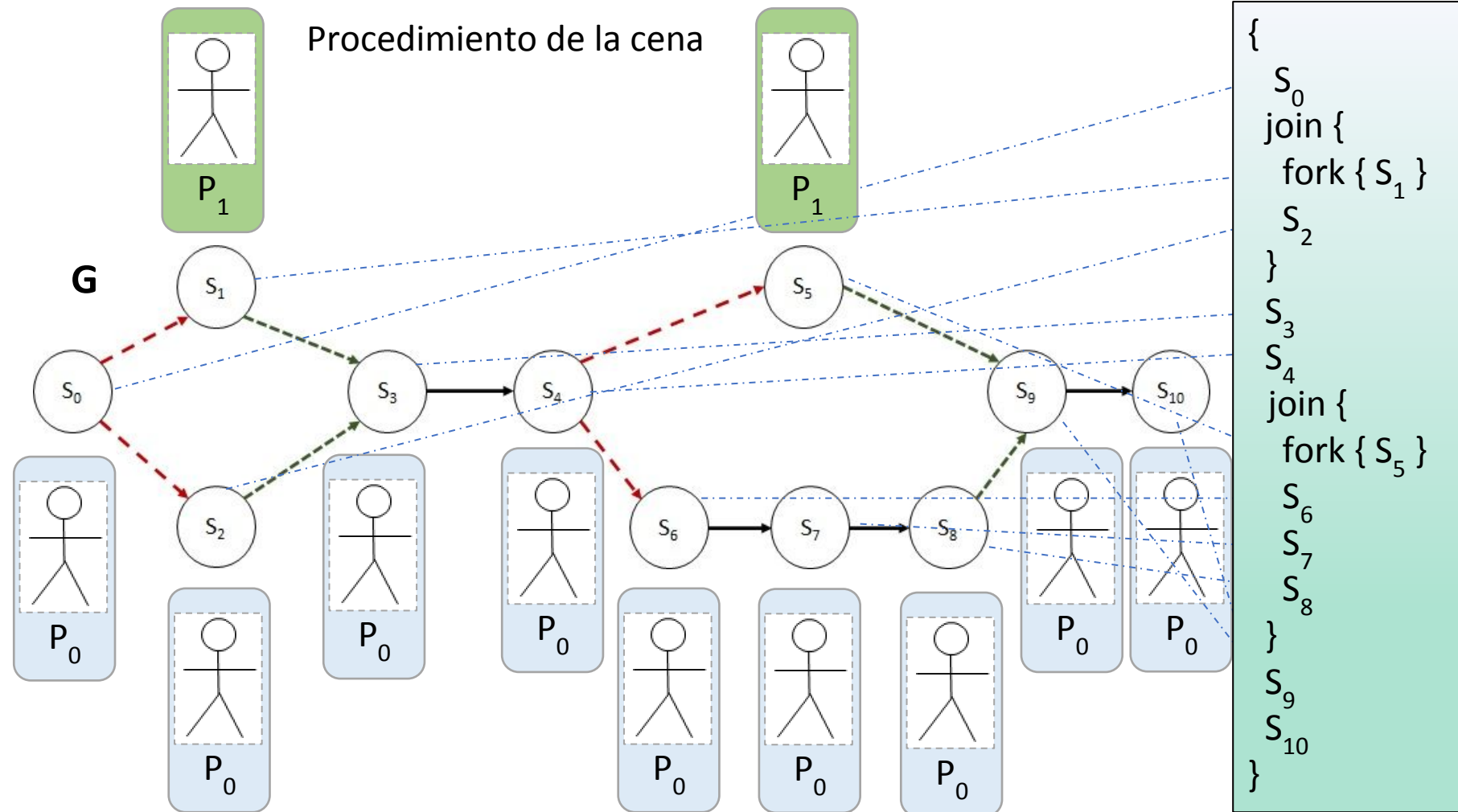
Inicio y terminación de tareas



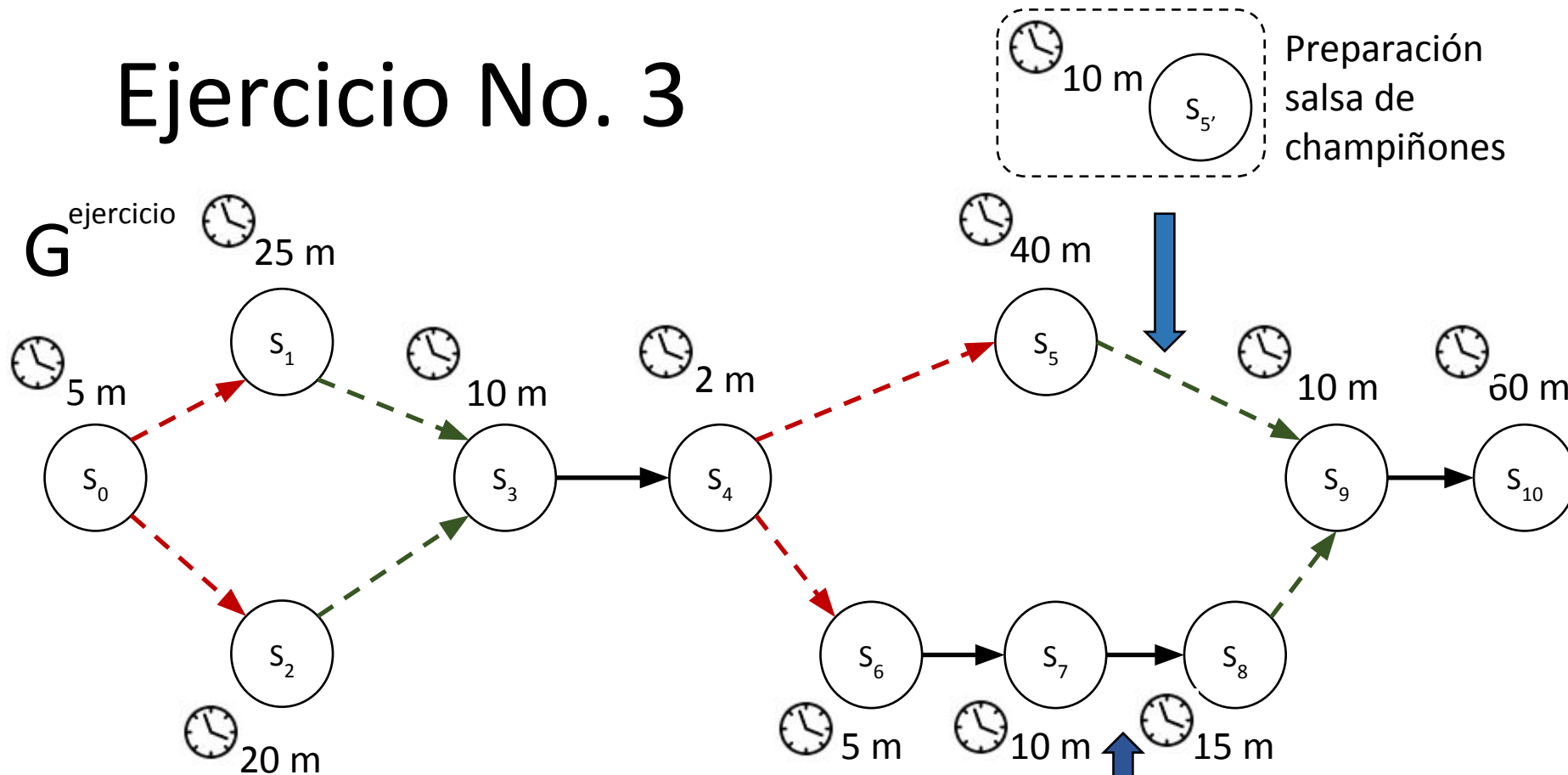
```
{  
  s1  
  join{  
    fork { s2 }  
    fork { s3 }  
    s4  
  }  
  s5  
  s6  
}
```

En algunos lenguajes no se utilizan las palabras `async` ni `finish`, en su lugar utilizan `fork` y `join`.

Inicio y terminación de tareas



Ejercicio No. 3



Para el grafo $G^{\text{ejercicio}}$ (el del ejercicio No. 2), luego de adicionarle las dos actividades indicadas en el diagrama, hacer el pseudocódigo de inicio y terminación de tareas con `async` y `finish`.

Ejercicio No. 4

- El siguiente pseudo-código suma dos matrices triangulares inferiores (matrices cuadradas $n \times n$ en las cuales los elementos por encima de la diagonal, incluyendo la diagonal de $(0,0)$ a (n,n) , son cero). En el código, cada ejecución de la sentencia $A[i][j] = B[i][j] + C[i][j]$; representa una unidad de tiempo en el grafo computacional correspondiente.

```
finish {  
  for (int i = 0; i < n; i++) {  
    async {  
      for (int j = 0; j < i; j++) {  
         $A[i][j] = B[i][j] + C[i][j]$ ;  
      } // for-j  
    } // async  
  } // for-i  
} // finish
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 \\ 5 & 4 & 0 & 0 \\ 8 & 7 & 1 & 0 \end{bmatrix}$$

- En términos de n ¿Cuál es el WORK del código mostrado?

Contenido

- Introducción
 - ¿Qué es paralelismo a nivel de tarea?
- Grafo computacional
- Paralelismo ideal
- Planificación (scheduling) en multiprocesadores
- Análisis de algoritmos paralelos
 - Ley/Argumento de Amdahl
 - Ley de Gustafson-Barsis
- Retos y Dificultades en la computación paralela
- Computación heterogénea
- Referencias

Análisis de algoritmos paralelos

Análisis de algoritmos paralelos

- El análisis del tiempo de ejecución de la computación utiliza los siguientes conceptos:
 - El trabajo (**work**) de una computación ejecutada por p procesadores es el número total de operaciones primitivas que realizan los procesadores. Ignorando el esfuerzo extra en Comunicaciones que se requiere para sincronizar los procesadores, este trabajo (*work*) es igual al tiempo utilizado para ejecutar la misma computación en un único procesador, descrito como T_1 .

Análisis de algoritmos paralelos

- El análisis del tiempo de ejecución de la computación utiliza los siguientes conceptos:
 - La extensión (**span**) de la computación es la serie de operaciones más larga que ha sido realizada secuencialmente debido a las dependencias de los datos (trayecto crítico). La extensión (*span*) también puede ser llamada la longitud del trayecto crítico o el ancho de la computación. Minimizar la extensión (*span*) es importante en el diseño de algoritmos paralelos, pues la extensión determina el tiempo de ejecución más corto posible.
 - El costo (**cost**) de la computación es la cantidad pT_p . Esta expresa el tiempo total gastado, por todos los procesadores, tanto procesando como esperando.

Análisis de algoritmos paralelos

- Con base en las definiciones de trabajo (*work*), extension (*span*) y costo (*cost*) se pueden describir la ley del trabajo (*work law*) y la ley de la extension (*span law*) para computación paralela
 - Ley del trabajo (*work law*): El costo siempre es mayor o igual al trabajo:
 $pT_p \geq T_1$. Esto se sigue del hecho que p procesadores pueden realizar máximo p operaciones en paralelo.
 - Ley de la extensión (*span law*): un número *fínito* p de procesadores NO puede superar un número infinito ∞ de procesadores, de tal forma que siempre
 $T_p \leq T_\infty$.

Mejora en la rapidez (*Speedup*)

- Es una medida de desempeño que muestra la ganancia en rapidez lograda mediante la ejecución paralela comparada con la ejecución secuencial: $S_p = T_1 / T_p$.
- Cuando la mejora en la rapidez es $\Omega(n)$ para una entrada de tamaño n (utilizando la notación big O), la mejora en la rapidez es lineal y es el comportamiento óptimo en modelos simples de computación pues la *ley del trabajo* implica $T_1 / T_p \leq p$
 - Mejora en la rapidez super-lineal puede ocurrir en la práctica debido a los efectos de la jerarquía de la memoria [memoria cache de alta velocidad].
- La situación $T_1 / T_p = p$ es llamada la mejora en la rapidez perfecta.
- Un algoritmo que exhibe mejora en la rapidez (*speedup*) lineal se dice que es escalable.

Otras medidas de desempeño

- Eficiencia (**Efficiency**) es la mejora en la rapidez (*speedup*) por cada procesador: S_p / p .
- Paralelismo (**Parallelism**) es la razón T_1 / T_∞ . Representa la máxima mejora en la rapidez (*speedup*) possible para cualquier número de procesadores. Por la ley de extensión, el paralelismo limita la mejora en la rapidez:
 - si $p > T_1 / T_\infty$, entonces $T_1 / T_p \leq T_1 / T_\infty < p$
- Inconstancia (**Slackness**): es la razón $T_1 / (pT_\infty)$. Una Inconstancia (*slackness*) menor que uno implica (por la ley de extensión) que una mejora en la rapidez lineal perfecta es imposible sobre p procesadores.



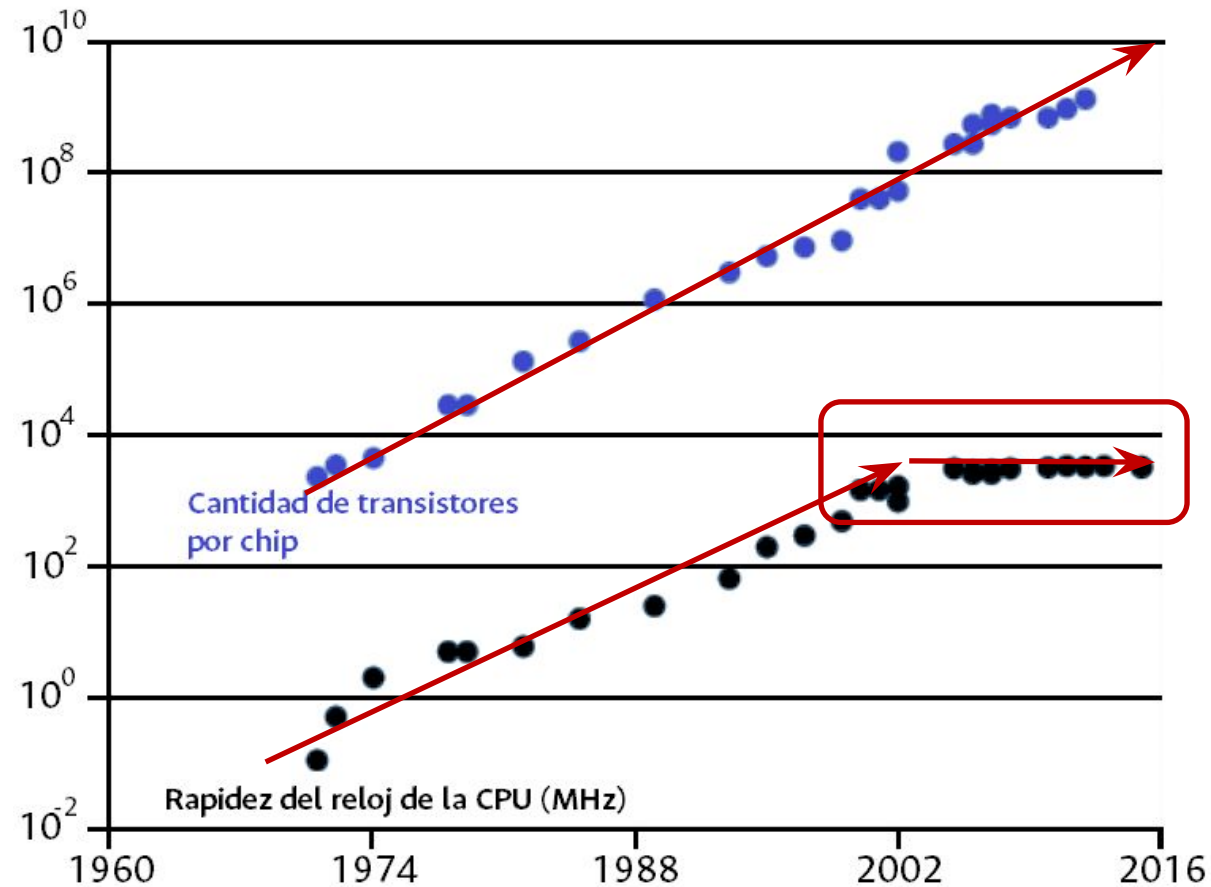
Ley de Moore

- En el año 1965, Gordon Moore escribió un artículo de tres páginas que describía dos asuntos:
 1. La consolidación del mercado de la electrónica hacia los circuitos integrados
 2. La capacidad de duplicar cada año el número de componentes en un circuito integrado por lo menos durante una década.
- En 1975, [10 años después] él revisó su predicción y ajustó la estimación a dos años para duplicar la cantidad de componentes colocados dentro de un circuito integrado.
 - David House: esta “duplicación” de la cantidad de transistores por circuito integrado se traduce en que un procesador puede duplicar su desempeño cada 18 meses sin incrementar el consumo de potencia.
- Esta predicción se mantuvo aproximadamente cierta hasta cerca del año 2012.

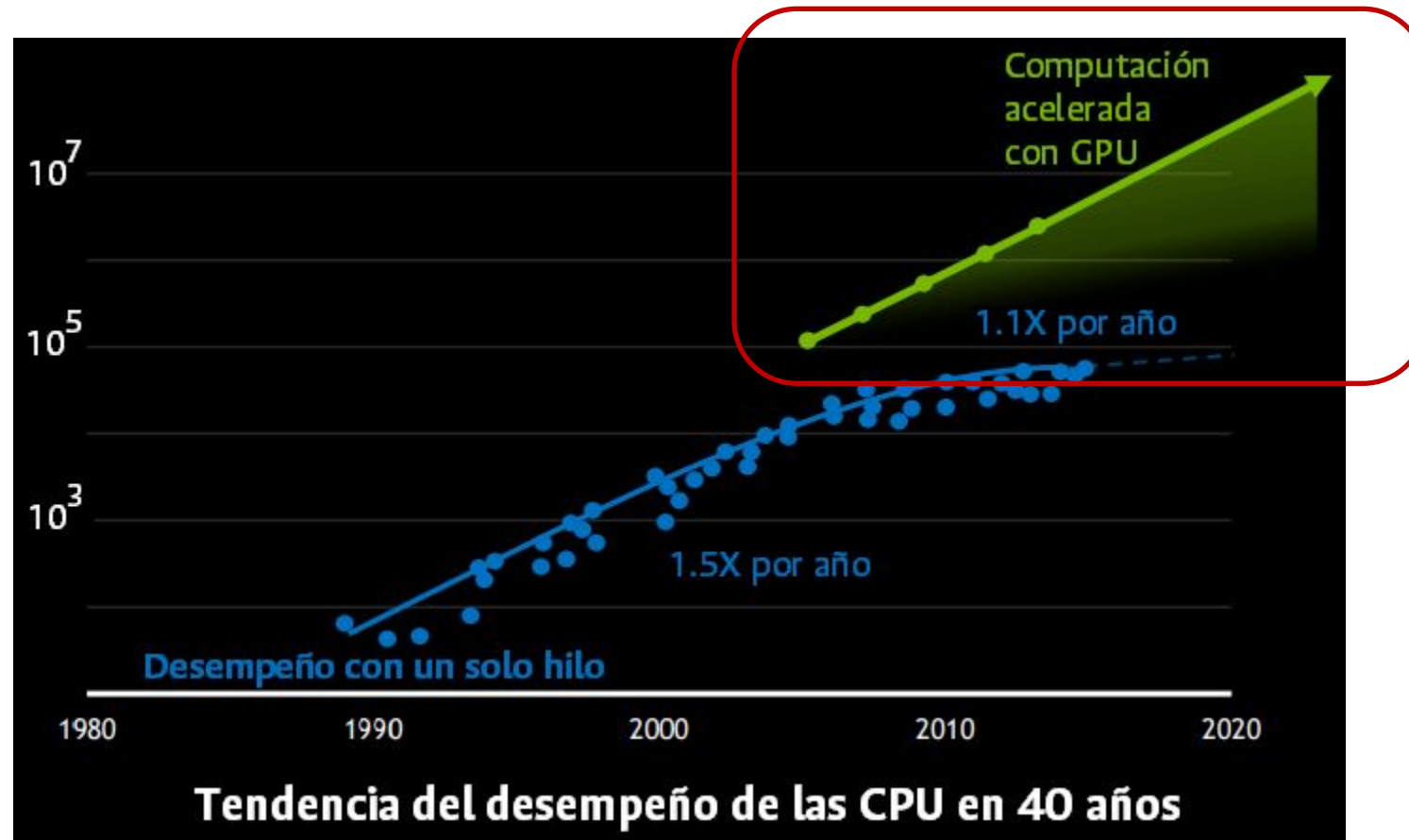
Cada dos años se duplicará la cantidad de componentes colocados en un circuito integrado



Transistores por chip y la rapidez de la CPU



Desempeño de las CPU y las GPU



No más ley de de Moore

- Varias compañías se anticiparon a esta reducción en la tasa de crecimiento que predecía la ley de Moore y comenzaron a investigar alternativas para incrementar la capacidad de cómputo.
- El resultado de de estos esfuerzos son los procesadores con multiples núcleos (cores)
- Esta parecía una solución adecuada para el problema de alcanzar el límite de la ley de moore, pero los científicos de la computación pronto se encontraron frente a los límites de otra ley: la ley de Amdahl, nombrada así por Gene Amdahl.

Opcional: Leer el capítulo **ARQUITECTURAS PARALELAS** del libro
“Computer Architecture. *From the Stone Age to the Quantum Age*”. de **Fox, C.** No Starch Press. 2024

Ley/Argumento de Amdahl

- En computación, es una fórmula que busca estimar la mejora teórica en el tiempo de ejecución que se logrará para una tarea que tiene asignada **una carga fija de trabajo**, dentro de un sistema cuyos recursos han sido mejorados.

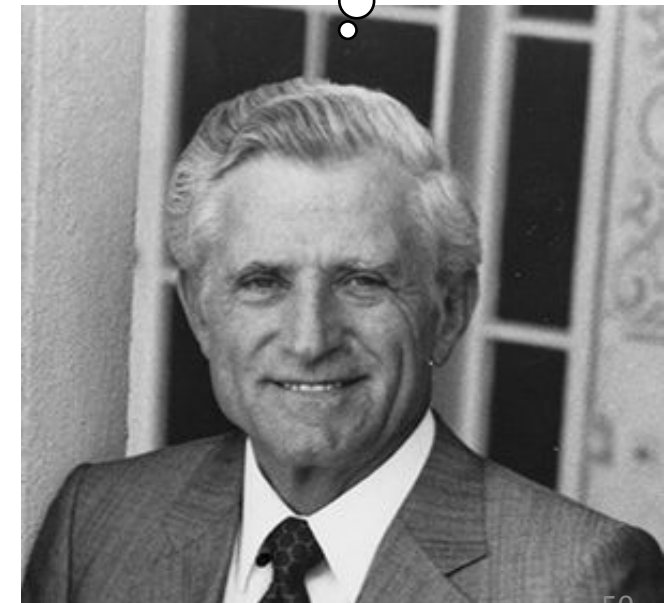
LA LEY DE AMDHAL ESTABLECE QUE LA MEJORA EN LA RAPIDEZ [SPEEDUP] TEÓRICA ESTÁ DETERMINADA POR LA FRACCIÓN DE CÓDIGO P QUE PUEDE SER PARALELIZABLE

$$\text{SPEEDUP} = \frac{1}{(1-P) + P/N}$$

PARTE SERIAL DE LA COMPUTACIÓN = $1(100\%) - \text{PARTE PARALELA}$

LA PARTE PARALELA ESTÁ REPARTIDA EN N PROCESADORES

$\text{SPEEDUP} \leq 1/q$



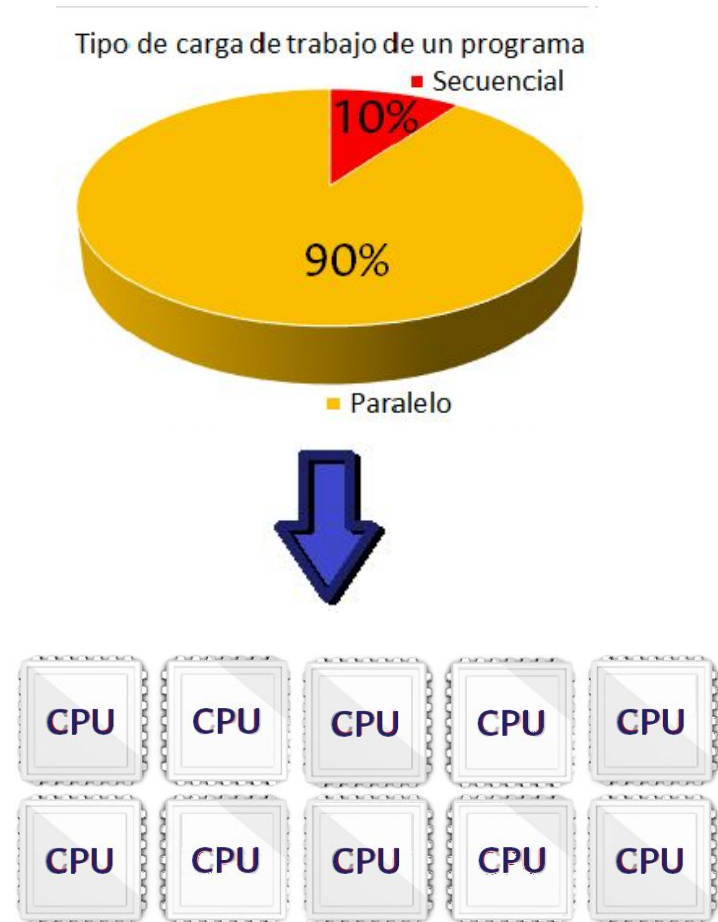
Ley/Argumento de Amdahl

- Sea q la fracción de la aplicación (carga de trabajo) que es secuencial.
- Amdahl propuso que la mejora máxima en la rapidez (*speedup*) está limitada por $1/q$:

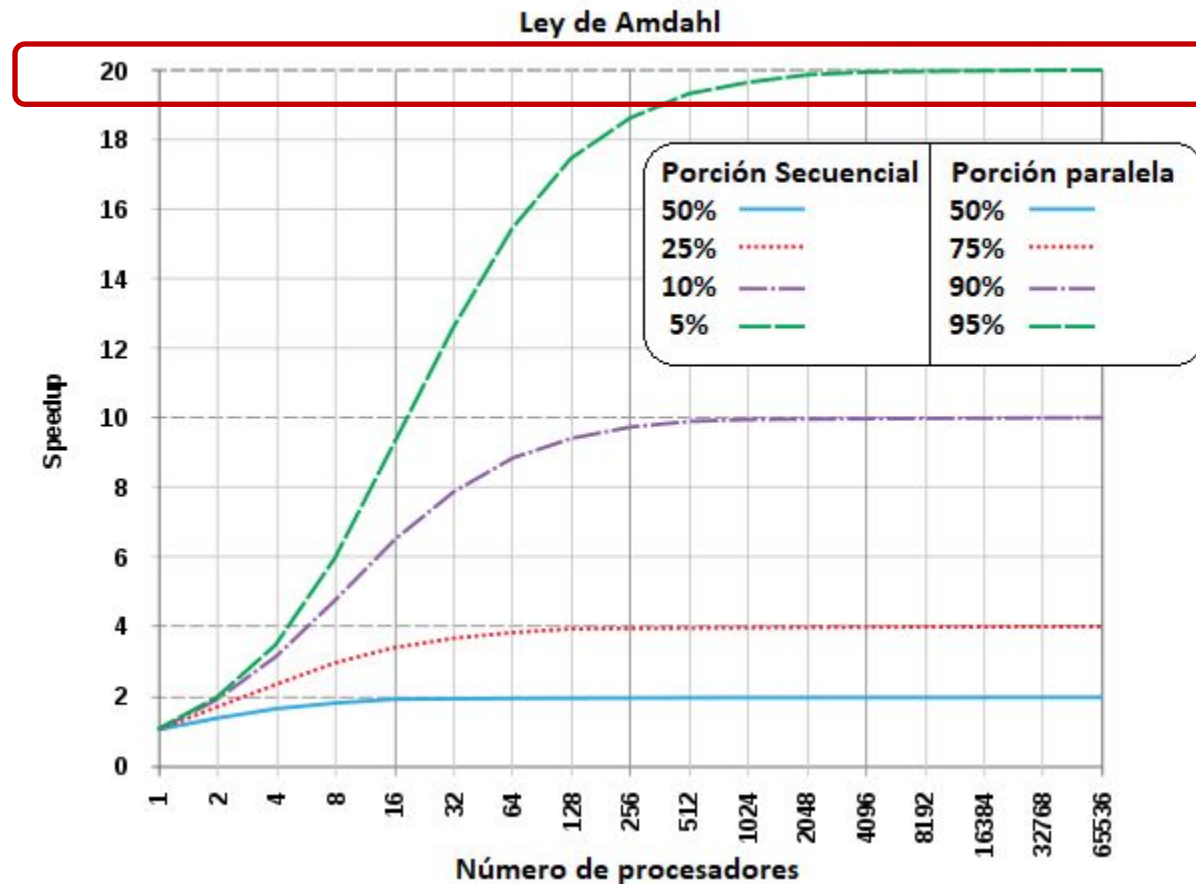
$$\text{SPEEDUP} \leq 1/q$$

Otro ejemplo, si el 10% de la carga de trabajo es obligatoriamente secuencial, es decir que $q = 0,1$; en este caso la mejora en la rapidez estará limitada a un valor máximo de 10 (sin importar la cantidad de procesadores o cores que se tengan):

$$\text{SPEEDUP} < 10$$

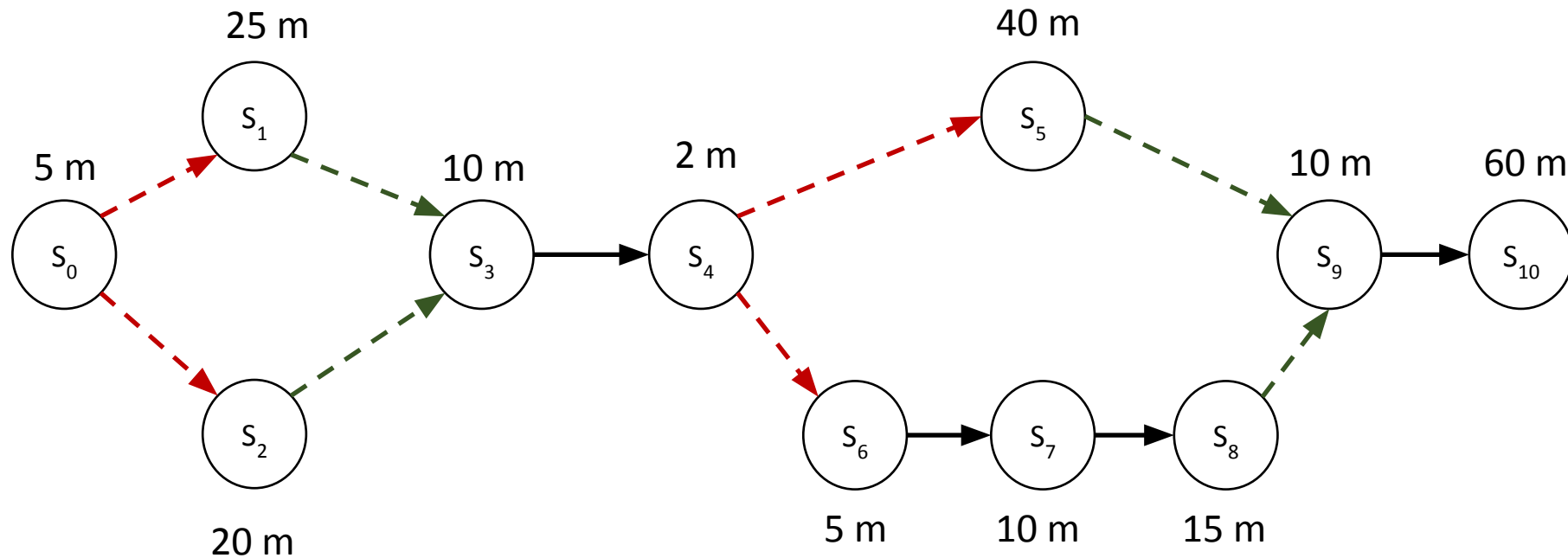


Ley/Argumento de Amdahl



Comportamiento del mejoramiento teórico de la rapidez de ejecución de un programa (*speedup*) de acuerdo con la ley de Amdahl en función de la cantidad de procesadores que lo ejecuten en paralelo.

Ley/Argumento de Amdahl



$$\frac{[\text{Work} - \text{Span}]}{\text{Work}} = \frac{[202 - 152]}{202} = 24,75\%$$

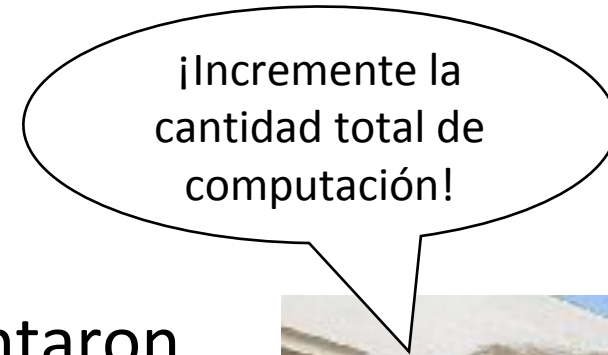
$$q = 1 - 0,2475 = 75,25\%$$

Ley/Argumento de Amdahl

- Cuando se diseña hardware o software, se habla del ***cuello de botella de la ley de Amdahl***: una parte de la computación es inherentemente secuencial y define un límite para la mejora en la rapidez (*speedup*) ✓
- Cuando se diseñen algoritmos paralelos hay que tener mucho cuidado con este comportamiento. ✓
- Si el algoritmo tiene un 10% de actividades secuenciales al inicio y al final, esto va a limitar la cantidad de paralelismo que se puede conseguir. ✓

Opcional: leer el capítulo 2 “Ejecución en serie y en paralelo” del libro **BOBROV, K. “Concurrency”. 2024 by Manning Publications Co.**

Ley de Gustafson-Barsis



- John L. Gustafson y Edwin H. Barsis la presentaron en 1988 en el artículo “*Reevaluating Amdahl's Law*”
- En computación, es una fórmula que busca estimar la mejora teórica en el tiempo de ejecución que se logrará para una tarea, que tiene asignado **un tiempo de ejecución fijo**, dentro de un sistema cuyos recursos han sido mejorados.



Las limitaciones impuestas por la parte secuencial de un programa pueden ser contrarrestadas al incrementar la cantidad total de computación.

Limitaciones de la Ley de Gustafson

- Algunos problemas de computación NO tienen conjuntos de datos grandes y NO podrán aprovechar las ventajas del paralelismo que se obtienen al tener en cuenta la ley de Gustafson.
- Los algoritmos con tiempo de ejecución NO lineales puede dificultárseles provechar el paralelismo descrito por la ley de Gustafson. Un algoritmo con $O(N^3)$ al duplicar la concurrencia obtendrá un incremento del 26% en el tamaño del problema. Es decir que aunque utilice muchos procesadores la ventaja NO será mucha con respecto al paralelismo utilizado originalmente (sin embargo, en problemas reales la mejora es importante).

Contenido

- Introducción
 - ¿Qué es paralelismo a nivel de tarea?
- Grafo computacional
- Paralelismo ideal
- Planificación (scheduling) en multiprocesadores
- Análisis de algoritmos paralelos
 - Ley/Argumento de Amdahl
 - Ley de Gustafson-Barsis
- Retos y Dificultades en la computación paralela
- Computación heterogénea
- Referencias

Retos y Dificultades en la computación paralela

Retos y Dificultades

- #01 Condiciones de carrera
- #02 Deadlock [abrazo mortal, punto muerto]
- #03 LiveLock [Ejecución sin progreso]
- #04 Starvation [Inanición]
- #05 Atomicidad
- #06 Sincronización de acceso a la memoria
- #07 Exclusión mutua y cerrojos (locks)
- #08 Escalamiento estrangulado (Strangled Scaling)
- #09 Falta de localidad
- #10 Desequilibrio de carga
- #11 Overhead [sobrecarga]
- #12 Las personas

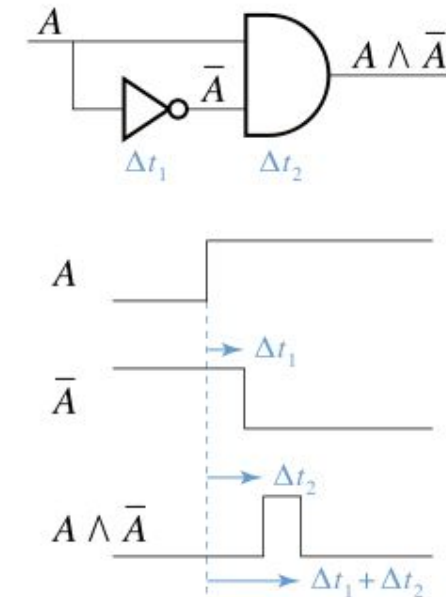


Condiciones de carrera

- Cuando la salida o estado de un proceso es dependiente de una secuencia de eventos que se ejecutan en orden arbitrario y van a trabajar sobre un mismo recurso compartido, se puede producir un error cuando NO se ejecutan en el orden que el programador esperaba.



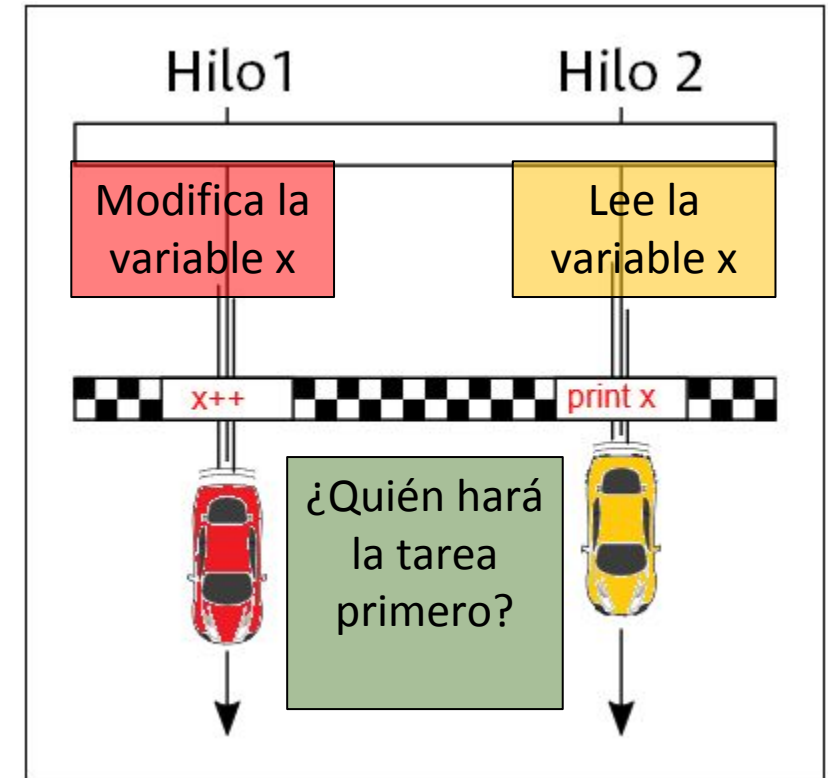
Condición de carrera en un circuito lógico.



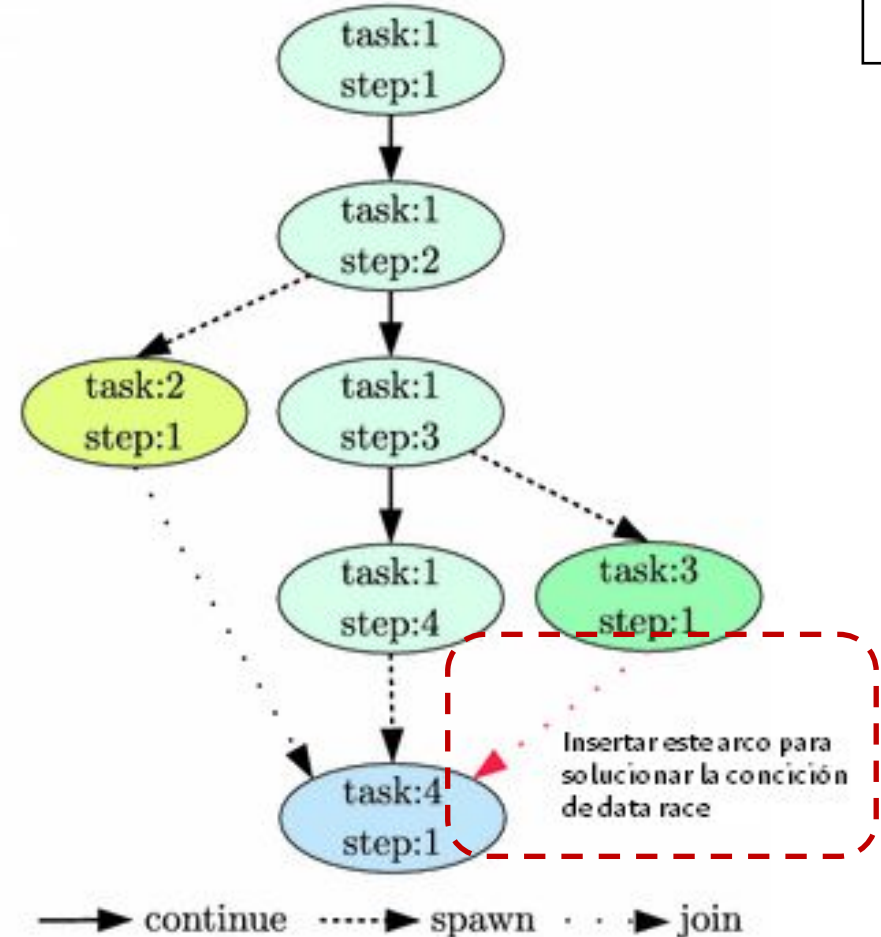
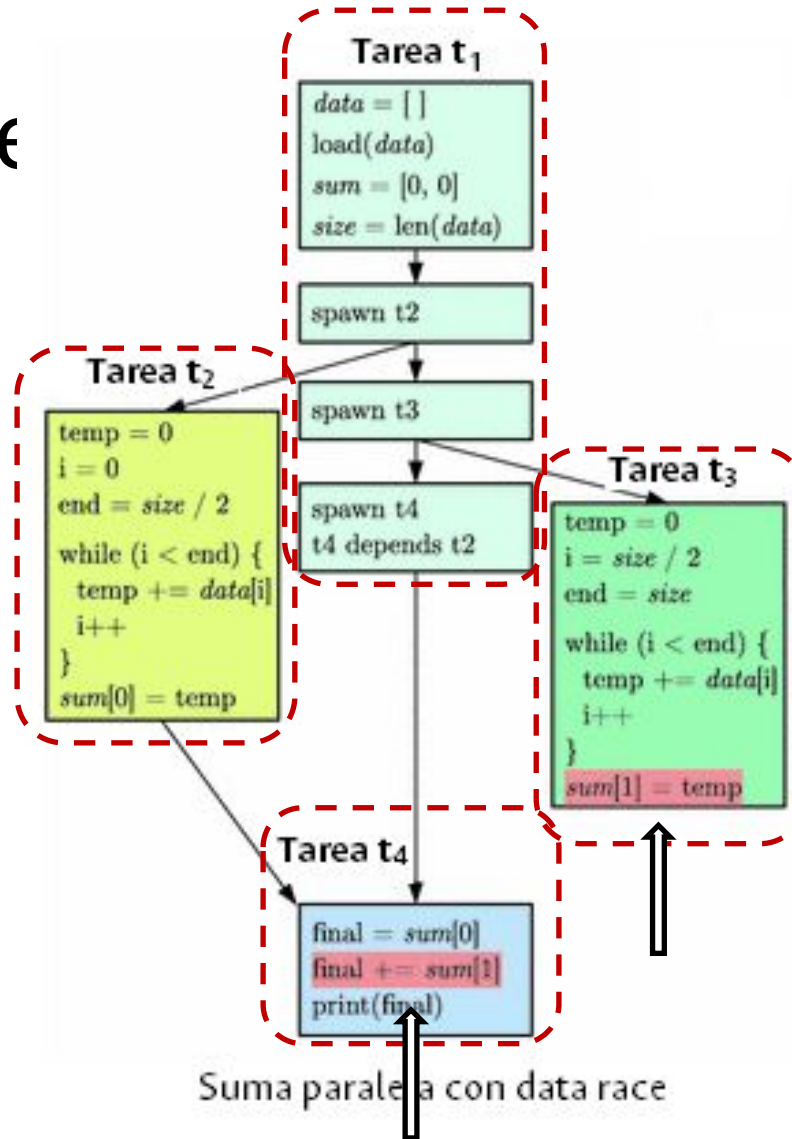
Δt_1 y Δt_2 representan el retardo de propagación de la señal de los elementos lógicos. Cuando el valor de entrada (A) cambia, el circuito genera en la salida un pico de duración $(\Delta t_1 + \Delta t_2) - \Delta t_2 = \Delta t_1$.

Data race

- Una condición de carrera en los datos puede ocurrir cuando dos hilos acceden la misma variable concurrentemente, y al menos uno de los accesos es de escritura.
- Se dice que la condición de Carrera para los datos (data race) ocurre en el lugar L dentro de un grafo computacional G, si existe un paso S_1 y un paso S_2 en G tal que NO hay un camino de arcos dirigidos desde S_1 hasta S_2 o desde S_2 hasta S_1 en G, y los dos pasos S_1 y S_2 leen o escriben L (con al menos uno de los accesos como escritura, ya que dos lecturas en paralelo NO presentan este problema).



Data race

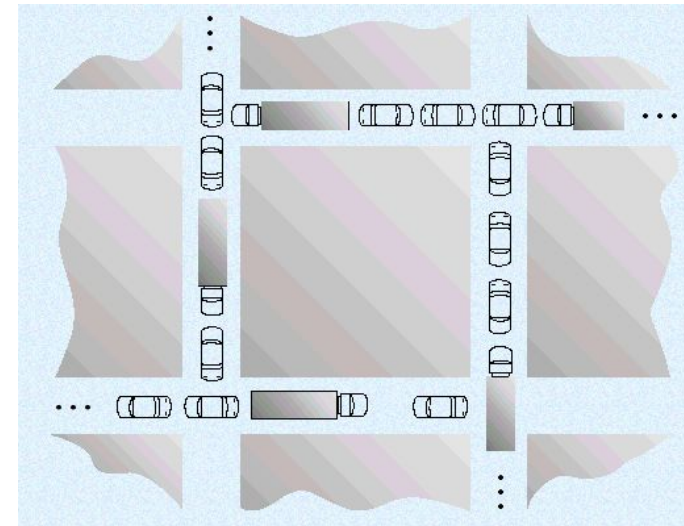
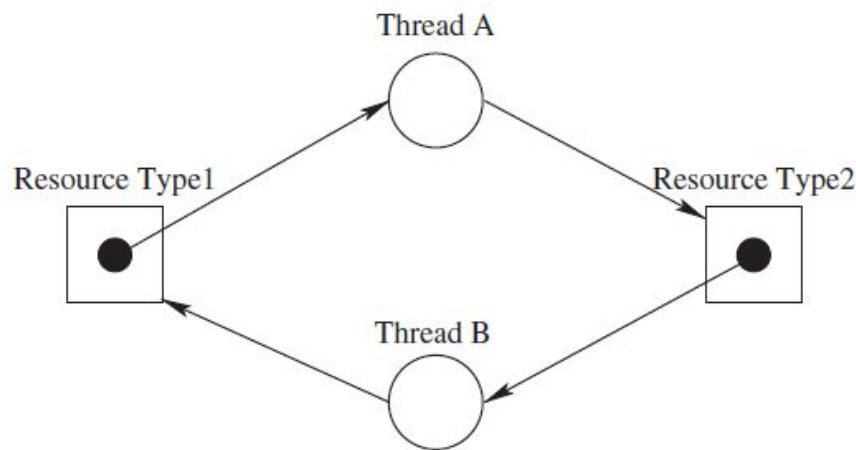


Grafo computacional del programa con data race

Opcional: leer el capítulo 8 “ Resolución de problemas de concurrencia: condiciones de carrera y sincronización ” del libro BOBROV, K. “Concurrency”. 2024 by Manning Publications Co.

Deadlock (abrazo mortal, punto muerto, bloqueo mútuo)

- Es un fenómeno que involucra multiples hilos (dos o más)
- Los hilos pueden quedarse esperando indefinidamente para conseguir acceso a algún tipo de recurso, mientras tienen asignados otros, que no liberarán hasta que tengan acceso al tipo de recurso esperado.



Modelando el sistema para deadlocks

- El Sistema está conformado por recursos ✓
- “Tipos” de recursos $R_1, R_2, R_3 \dots, R_m$ ✓
 - Ciclos de CPU
 - Espacio en memoria
 - Dispositivos de E/S
- Cada tipo de recurso R_i tiene W_i instancias. ✓
- Para que un proceso utilice un recurso debe realizar tres pasos: ✓
 1. Solicitarlo (request)
 2. Utilizarlo (use)
 3. Liberarlo (release)

Caracterización de los Deadlock

Un deadlock puede ocurrir si se tienen, de manera simultánea, las cuatro condiciones siguientes (condiciones de Coffman)

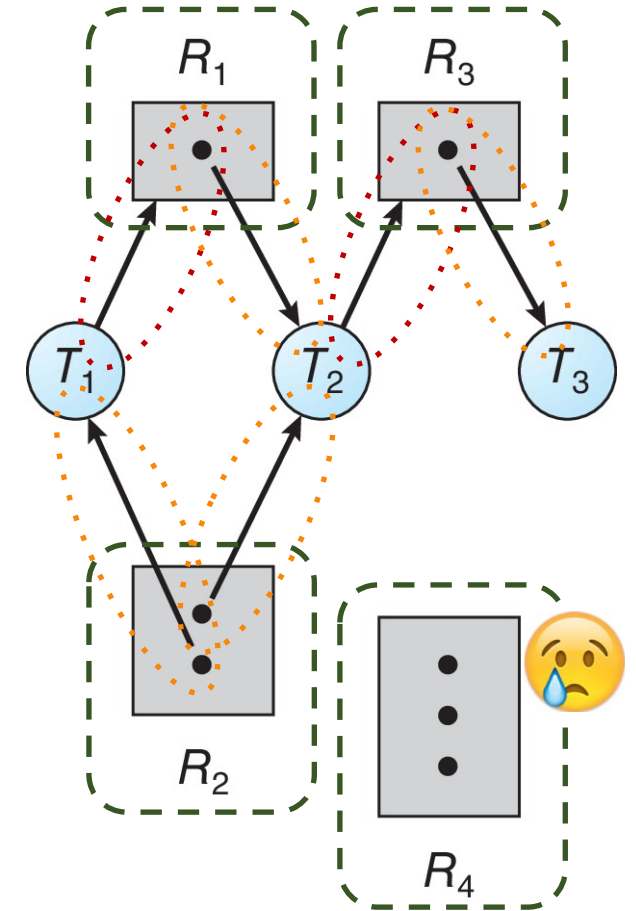
- **Exclusión mutua:** solo un proceso, a la vez, puede utilizar un recurso
- **Ocupar y esperar (Hold and wait):** un proceso ocupando al menos un recurso está esperando para acceder a un recurso adicional que está siendo usado por otro proceso
- **Sin derecho preferente (No preemption):** un recurso puede ser liberado solo de manera voluntaria por el proceso que lo utilizando, luego de que el proceso haya terminado su tarea.
- **Espera circular (Circular wait):** hay un conjunto $\{P_0, P_1, \dots, P_n\}$ de procesos esperando, tal que P_0 está a la espera por un recurso que está siendo ocupado por P_1 , P_1 está aguardando por un recurso que está usando por P_2 , ..., P_{n-1} está esperando por un recursos que está siendo utilizado por P_n , y, finalmente, P_n está esperando por un recurso que está siendo ocupado por el proceso P_0 .

Grafo de asignación de recursos

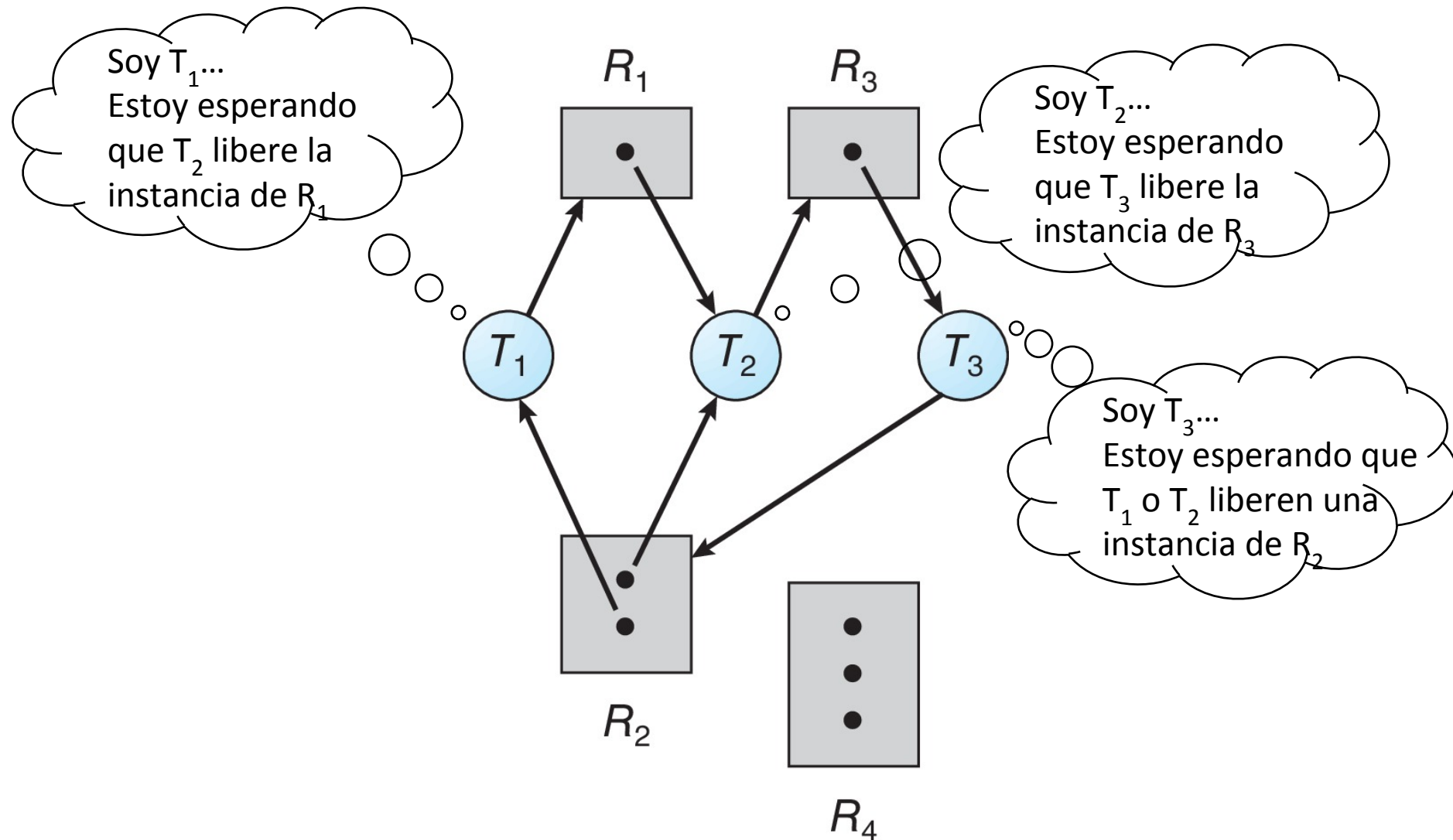
- Conformado por un conjunto de vértices V y un conjunto de arcos E .
- El conjunto V está particionado en dos clases de vértices:
 - $P = \{P_1, P_2, \dots, P_n\}$, el conjunto de todos los procesos en el Sistema
 - $R = \{R_1, R_2, \dots, R_m\}$, el conjunto de todos los “tipos de recursos” en el sistema
- **Arco de solicitud** – arco dirigido que va desde un proceso (quien solicita) hacia un recurso: $P_i \rightarrow R_j$
- **Arco de asignación** – arco dirigido que va desde un recurso (asignado) hacia un proceso $R_j \rightarrow P_i$

Ejemplo

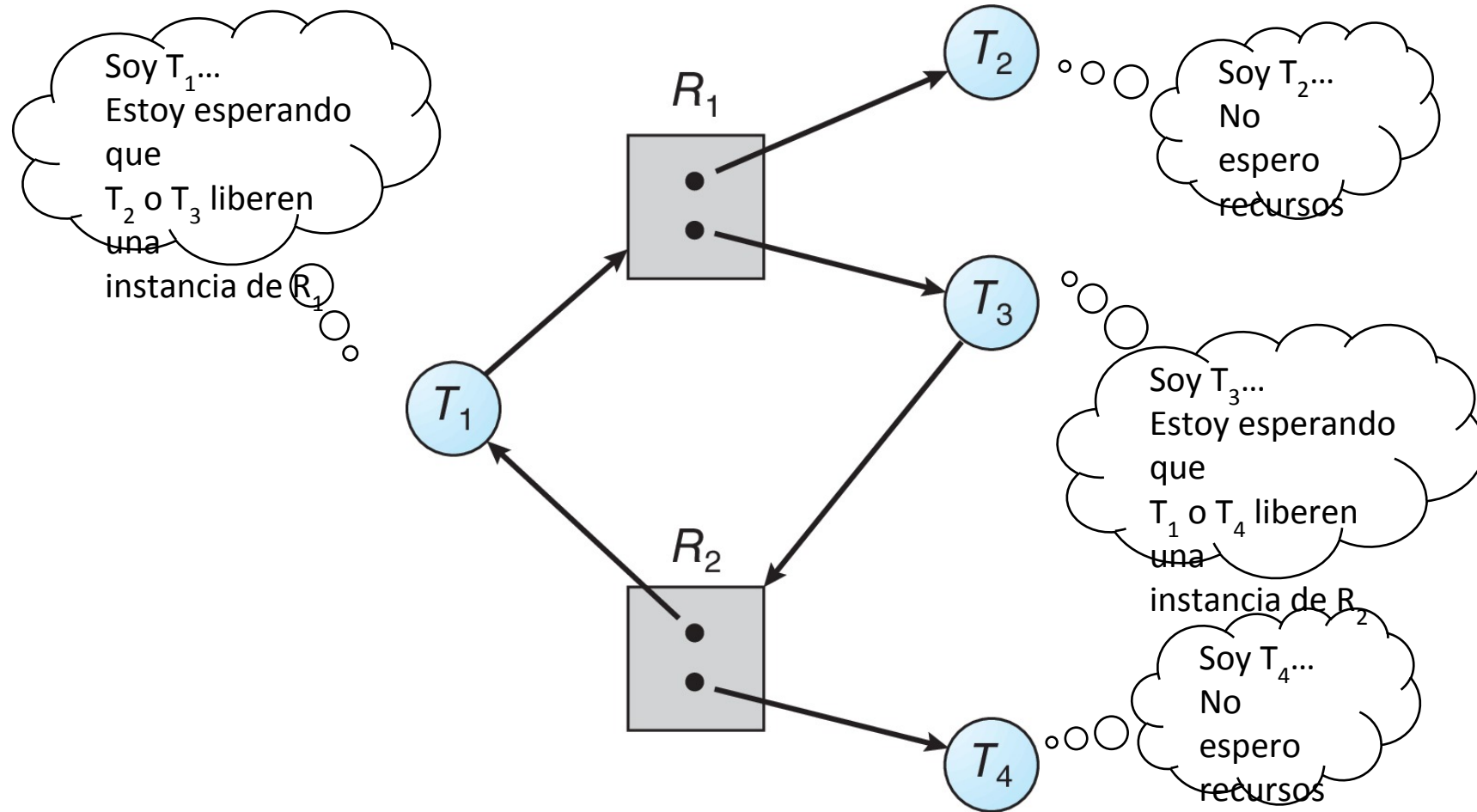
- Qué tenemos: ✓
 - Una instancia de recursos tipo 1 (R_1)
 - Dos instancias de recursos tipo 2 (R_2)
 - Una instancia de recursos tipo 3 (R_3)
 - Tres instancias de recursos tipo 4 (R_4)
- Qué muestra el grafo: ✓
 - La tarea T_1 tiene asignada una instancia de R_2 y está esperando por una instancia de R_1
 - La tarea T_2 tiene asignada una instancia de R_1 , una instancia de R_2 , y está esperando por una instancia de R_3
 - La tarea T_3 está utilizando una instancia de R_3
 - Ninguna tarea tiene asignados recursos de R_4



Asignación de recursos CON deadlock



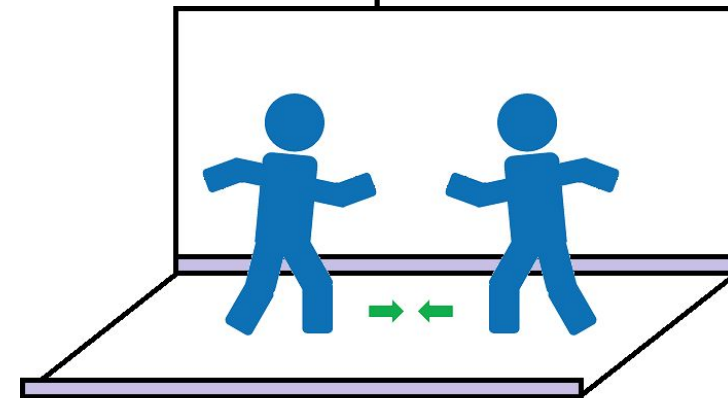
Asignación de recursos SIN deadlock



Opcional: leer el capítulo 8 “**Resolución de problemas de concurrencia: interbloqueos e inanición**” del libro
BOBROV, K. “Concurrency”. 2024 by Manning Publications Co.

Ejecución sin progreso (LiveLock)

- Livelock no es deadlock
- Livelock ocurre cuando un proceso continua ejecutando pero no tiene la capacidad de progresar
- En la práctica, Livelock puede ocurrir debido a que el programador ha hecho cosas para evitar el deadlock, pero no se coordina con los otros procesos.
- Livelocks le ocurren a los programas que realizan actividades concurrentes de manera activa, pero en la práctica solo cambian el estado del proceso.



Ejecución sin progreso (Live

- Livelock puede ilustrarse con la función `pthread_mutex_trylock()` de Pthreads, que intenta adquirir un bloqueo mutex sin bloquearse.
- El ejemplo de código utiliza `pthread_mutex_trylock()`.
- Esta situación puede llevar a un livelock si `thread_one` adquiere `first_mutex`, seguido de `thread_two` adquiriendo `second_mutex`.
- Cada hilo invoca entonces `pthread_mutex_trylock()`, que falla, libera sus respectivos bloqueos, y repite las mismas acciones indefinidamente.

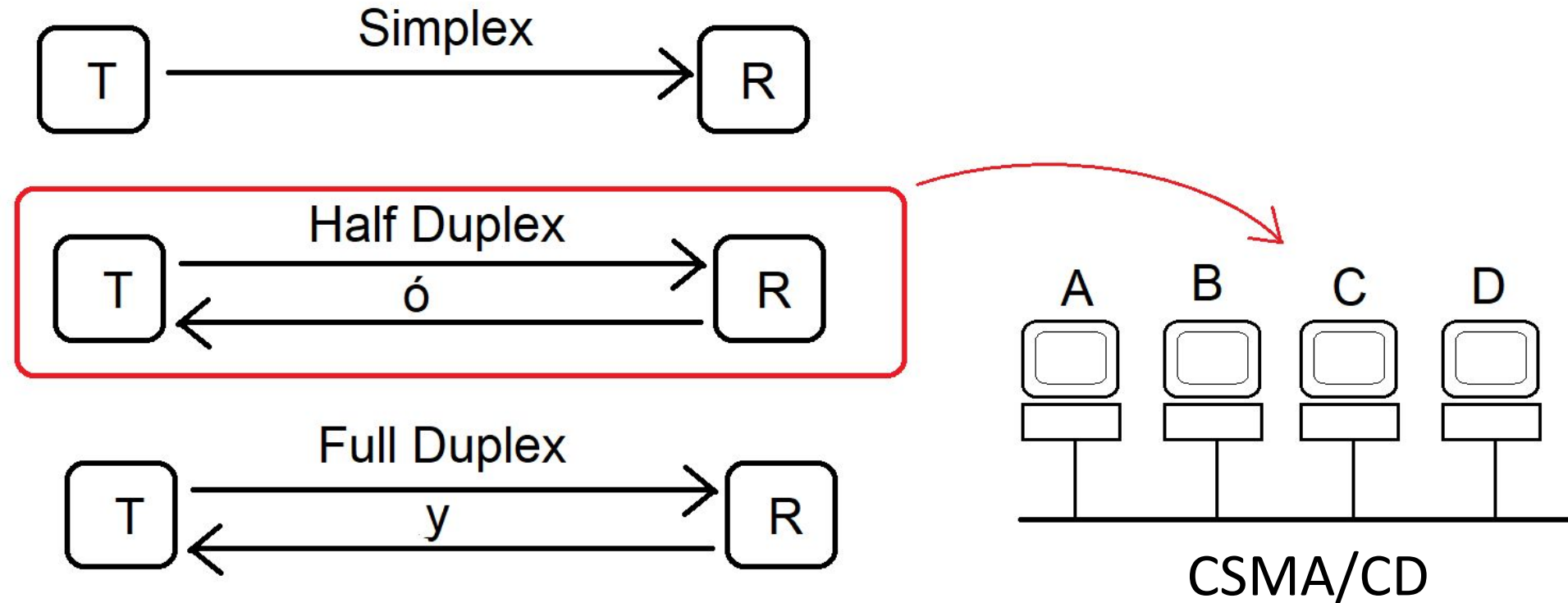
```
/* Hilo 01 se ejecuta en esta función */
void *do_work_one(void *param)
{
    int done = 0;
    while (!done) {
        pthread_mutex_lock(&first_mutex);
        if (pthread_mutex_trylock(&second_mutex)) {
            /**
             * Realiza alguna tarea
             */
            pthread_mutex_unlock(&second_mutex);
            pthread_mutex_unlock(&first_mutex);
            done = 1;
        }
        else
            pthread_mutex_unlock(&first_mutex);
    }
    pthread_exit(0);
}

/* Hilo 02 se ejecuta en esta función */
void *do_work_two(void *param)
{
    int done = 0;
    while (!done) {
        pthread_mutex_lock(&second_mutex);
        if (pthread_mutex_trylock(&first_mutex)) {
            /**
             * Realiza alguna tarea
             */
            pthread_mutex_unlock(&first_mutex);
            pthread_mutex_unlock(&second_mutex);
            done = 1;
        }
        else
            pthread_mutex_unlock(&second_mutex);
    }
    pthread_exit(0);
}
```

Ejecución sin progreso (LiveLock)

- El livelock suele producirse cuando los subprocesos reintentan operaciones fallidas al mismo tiempo. Por lo tanto, puede evitarse generalmente haciendo que cada subproceso reintente la operación fallida en momentos aleatorios. Este es precisamente el enfoque que adoptan las redes Ethernet cuando se produce una colisión de red. En lugar de intentar retransmitir un paquete inmediatamente después de que se produzca una colisión, un host implicado en una colisión dejará pasar un periodo de tiempo aleatorio antes de intentar transmitir de nuevo.
- El livelock es menos común que el deadlock, pero no deja de ser un problema a la hora de diseñar aplicaciones concurrentes y, al igual que el deadlock, sólo puede producirse en determinadas circunstancias de programación.

Livelock y Ethernet Half Duplex



Opcional: leer el capítulo 8 “**Resolución de problemas de concurrencia: interbloqueos e inanición**” del libro BOBROV, K. “Concurrency”. 2024 by Manning Publications Co.

Inanición (Starvation)

- La inanición ocurre cuando la ejecución de un hilo o de un proceso es suspendida o deshabilitada por una cantidad de tiempo indefinida, aunque éste sea capaz de continuar su ejecución.
- La inanición está típicamente asociada con el cumplimiento estricto de las prioridades o por la carencia de justicia [en la planificación o en el acceso a los recursos]
- De algún modo, La inanición es un subconjunto de los livelocks.
- La inanición normalmente implica que hay uno o más procesos concurrentes codiciosos que no permiten que otros procesos puedan hacer su trabajo de la manera eficiente
- Una manera de identificar la inanición es utilizar alguna métrica. La inanición es un argumento excelente para hacer registros y muestreo de métricas.
 - Una de las formas en que se puede detectar y resolver la inanición es registrar cuándo un trabajo es terminado y luego determinar si la tasa de trabajo es la esperada



Atomicidad



- Cuando algo es considerado atómico [que no puede ser dividido] o que tiene la propiedad de atomicidad, significa que dentro del contexto que está operando es indivisible o no se puede interrumpir. Es como el funcionamiento del corazón: todo o nada, o funciona o no funciona.
 - Para el contexto de una aplicación, atomicidad significa que una transacción solo tiene dos opciones: (1) se ejecuta completa o (2) no se ejecuta.
- Importante: la palabra contexto
 - Un conjunto de instrucciones puede ser atómico en un contexto pero NO en otro.
 - La atomicidad de una operación puede cambiar dependiendo del ámbito que esté definido en ese momento
- La atomicidad puede trabajar a favor o en contra de la concurrencia



Sincronización de acceso a la memoria

- Supongamos que dos procesos están tratando de acceder la misma área memoria y la forma en que lo hacen no garantiza atomicidad ✓
- El resultado de esta actividad será NO determinística. ✓
- El conjunto de instrucciones de un programa que necesita acceso exclusivo a un recurso compartido es llamado “sección crítica”. ✓
- En memoria compartida. la gestión de la sección crítica exige sincronizar el acceso a la memoria . ✓
 - Esta labor exigirá por parte del desarrollador mantenimiento posterior al código y, además, puede afectar el desempeño de la ejecución del código.
- Tenemos aquí dos preguntas (nada fáciles de responder): ✓
 - ¿Las secciones críticas entrarán entrando y saliendo?
 - ¿Cuál es el tamaño recomendado para una sección crítica?

Problema de la sección crítica

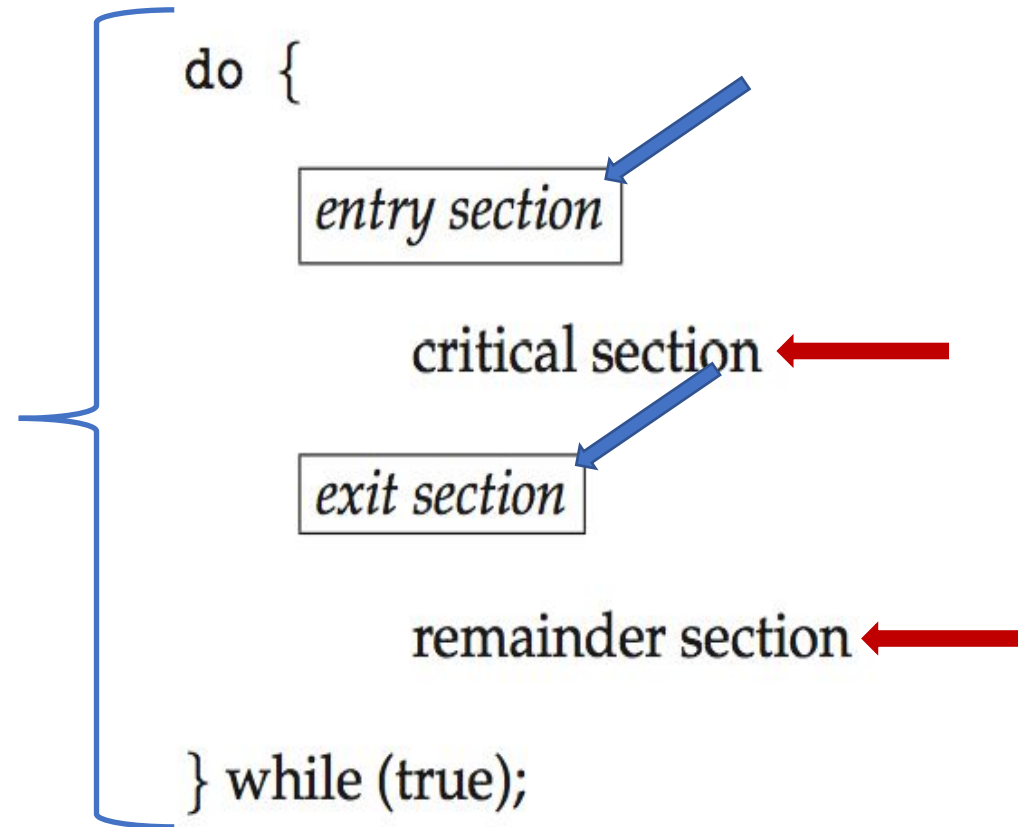
#06

- Considerese un sistema con n procesos $\{p_0, p_1, \dots, p_{n-1}\}$
- Cada proceso tiene un segmento de código especial llamado **sección crítica (critical section)**
 - Los procesos pueden cambiar variables comunes, actualizar tablas comunes, escribir en archivos comunes, etc.
 - Cuando un proceso está ejecutando el código de su sección crítica, *ningún otro proceso puede ejecutar su correspondiente sección crítica*.
- **El problema de la sección crítica** es diseñar un protocolo para garantizar esta condición en el sistema operativo.
- Cada proceso debe solicitar permiso para entrar en su sección crítica en la sección de entrada (**entry section**), si el sistema operativo se lo permite, ejecuta el Código de la sección crítica hasta llegar a la sección de salida (**exit section**), después de esto puede ejecutar el código restante (**remainder section**)

Sección crítica

#06

- Estructura general del proceso P_i



Problema de la sección crítica (Cont.)

#06

Requerimientos para resolver el problema de la sección crítica

1. **Exclusión mútua (Mutual Exclusion)** – Si el proceso P_i está ejecutando en alguna de sus secciones críticas, entonces ningún otro proceso puede estar ejecutando en sus correspondientes secciones críticas
2. **Progreso (Progress)** – Si ningún proceso está ejecutando en una sección crítica y hay algún proceso que dese entrar en su sección crítica, entonces la selección del siguiente proceso que entrará en la sección crítica no puede posponerse indefinidamente
3. **Espera acotada (Bounded Waiting)** – Debe existir un límite para la cantidad de veces que a otros procesos se le permita entrar a sus secciones críticas después que un proceso haya hecho una solicitud para entrar a su sección crítica de tal manera que se realmente se atienda dicha solicitud
 - Se está suponiendo que cada proceso se ejecuta con una rapidez diferente de cero
 - NO hay ninguna suposición en relación con la rapidez relativa (**relative speed**) de los n procesos

Opcional: leer el capítulo 8 “ Resolución de problemas de concurrencia: condiciones de carrera y sincronización ” del libro BOBROV, K. “Concurrency”. 2024 by Manning Publications Co.

Exclusión mutua y cerrojos (locks)

- Los cerrojos (locks) son una **solución de bajo nivel** para resolver el problema de la sección crítica y para eliminar condiciones de carrera
- Uno de los cerrojos básicos es el cerrojo mutex (mutex lock)
 - Una variable booleana indicando si el cerrojo (lock) está disponible o no: exclusión mútua.
- Un Seguro mutex protege una sección crítica.
 - Primero adquiere [`acquire()`] un cerrojo (lock)
 - Luego libera [`release()`] el cerrojo (lock)
- Las llamadas a las funciones `acquire()` y `release()` debe
 - Usualmente implementadas mediante instrucciones atómicas hardware.



Sección crítica usando cerrojo Mutex

```
while (true) {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
}
```



Ejemplo

Task A	Task B
<code>a = X;</code>	<code>b = X;</code>
<code>a += 1;</code>	<code>b += 2;</code>
<code>X = a;</code>	<code>X = b;</code>

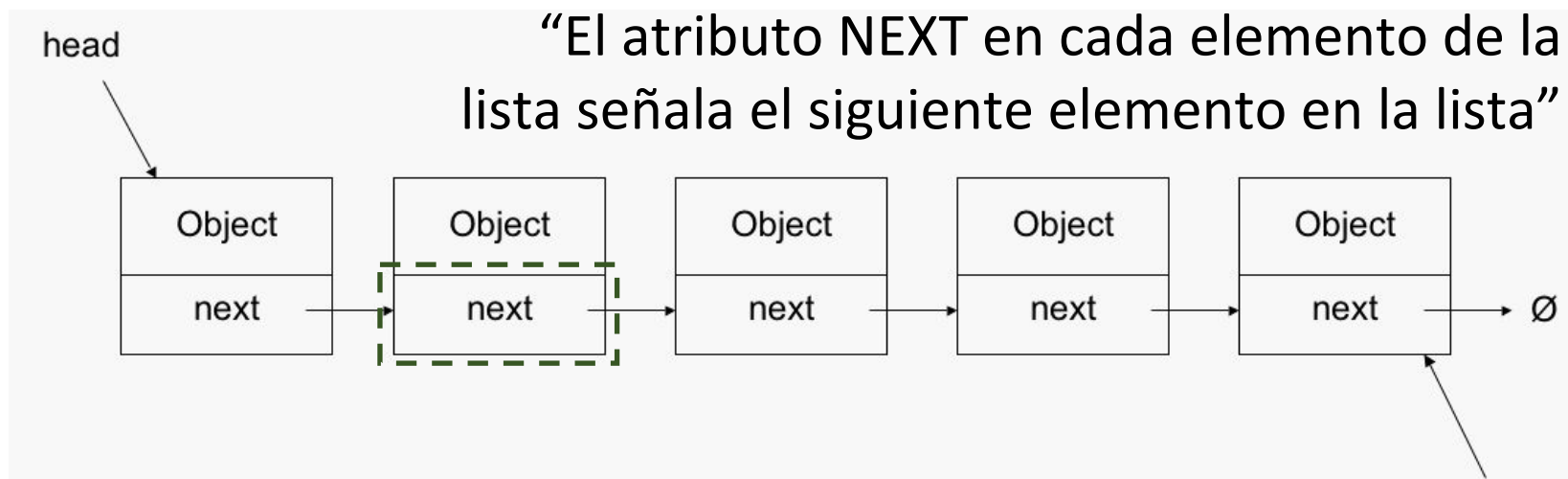
- Tarea A, adiciona 1 a X
- Tarea B, adiciona 2 a X
- La intención final es adicionar 3 a X
- Pero si hay falta de sincronización, las dos tareas pueden leer X al mismo tiempo y, dependiendo quien haga la última instrucción, solo se sumará 1, o solo se sumará 2.

Task A	Task B
<code>extern tbb::mutex M;</code>	<code>extern tbb::mutex M;</code>
<code>M.lock();</code>	<code>M.lock();</code>
<code>a = X;</code>	<code>b = X;</code>
<code>a += 1;</code>	<code>b += 2;</code>
<code>X = a;</code>	<code>X = b;</code>
<code>M.unlock();</code>	<code>M.unlock();</code>

- El Mutex M es conocido por las dos tareas.
- La primera tarea que adquiere el Mutex (le coloque el cerrojo) realizará su trabajo y la otra tarea debe esperar hasta que la primera tarea libere el mutex (le quite el cerrojo)
- Siempre se tendrá el resultado esperado

Los mutex protegen invariantes lógicas

- En el ejemplo anterior, la invariante lógica que se protege es “El valor de la variable X es la suma de los valores Adicionados”
- El mutex agrupa operaciones y estas pueden ser tratadas esencialmente como una única operación
- Con estructuras de datos, el mutex protegerá una invariante lógica asociada a varias ubicaciones de memoria.



Sincronización de grano grueso y de grano fino

- Se debe lograr balance entre
 - Sincronización de grano grueso: para el desempeño de la computación
 - Sincronización de grano fino: para el trato “justo”
- En OpenMP, una API para programación multiproceso con memoria compartida con base en el modelo de ejecución fork-join, tiene un pragma para identificar las secciones críticas y otro pragma para las operaciones atómicas.
 - `#pragma omp atomic`
 - `#pragma omp critical`



Escalamiento estrangulado (Strangled Scaling)

- Un mutex serializa la ejecución y adiciona un POTENCIAL cuello de botella tipo Amdhal.
- También se tendrán costos en la comunicación entre los núcleos.
- Si las tareas rara vez contienden por el mismo mutex, el impacto del mutex sobre escalamiento es muy poco.
- La técnica de “cerrojo de grano fino” (fine-grain locking) reemplaza un cerrojo (lock) que tiene alta contención con varios seguros (locks) que no tienen contención mejorando la escalabilidad.
- Se pueden utilizar operaciones atómicas en lugar de mutex cuando las invariantes lógicas afectan una única ubicación de memoria.

Falta de localidad

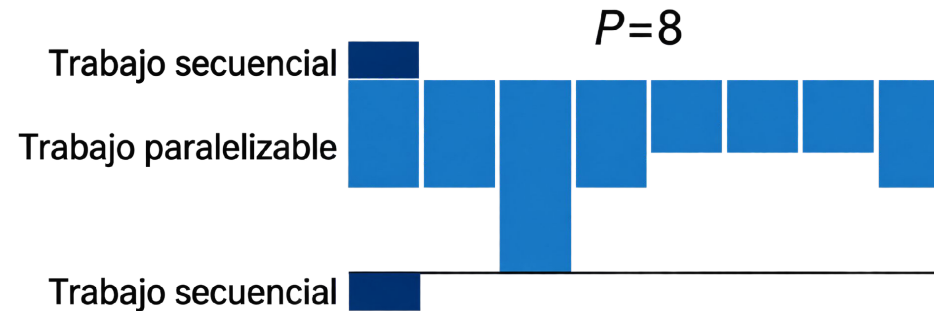
- La localidad es el otro factor clave del escalamiento
- La localidad hace referencia a dos apuestas sobre el acceso futuro a la memoria después que un núcleo accede a una locación
 - **Localidad temporal:** es probable que el núcleo acceda nuevamente a la misma locación en el futuro cercano.
 - **Localidad espacial:** es probable que el núcleo acceda a locaciones cercanas en el futuro cercano

Falta de localidad

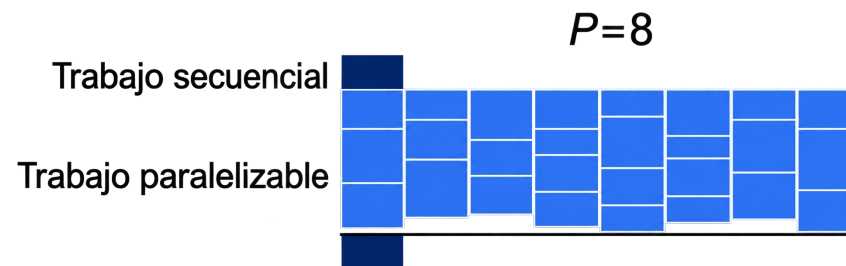
- Los programas deben escribirse para que procesen los datos completamente antes de pasar a procesar otros datos. Esto aumenta la cantidad de veces que una copia de los datos se encontrará en la memoria caché
 - **Cache oblivious algorithms**
[https://en.wikipedia.org/wiki/Cache-oblivious_algorithm]
- La comunicación es tan costosa y la computación tan barata que a veces vale la pena aumentar el trabajo a cambio de reducir la comunicación.
 - **Communication avoiding algorithms**
[https://en.wikipedia.org/wiki/Communication-avoiding_algorithm]

Desequilibrio de carga

- El desequilibrio de carga es la distribución desigual de trabajo entre todos los núcleos.

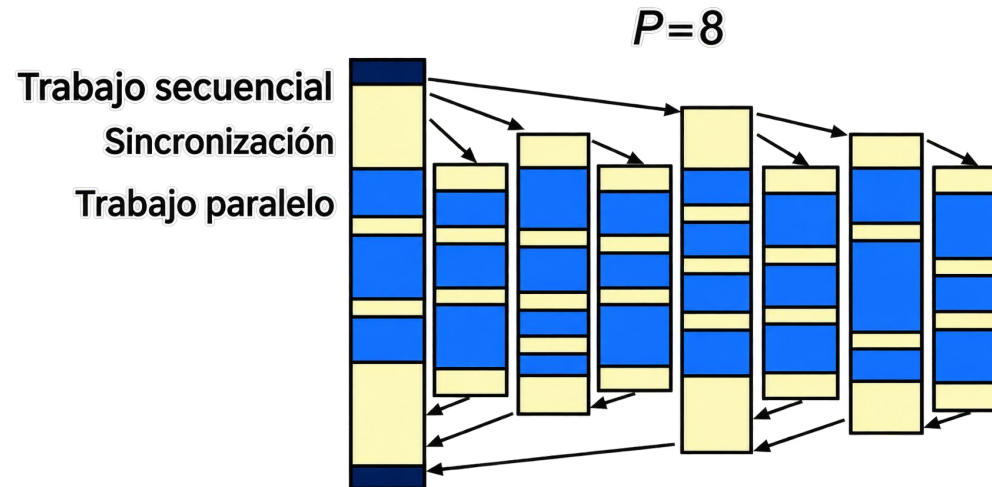


- La descomposición de tareas en una mayor cantidad que la cantidad de núcleos mejora las estimaciones de los tiempos de ejecución para la porción paralela.



Sobrecarga (Overhead)

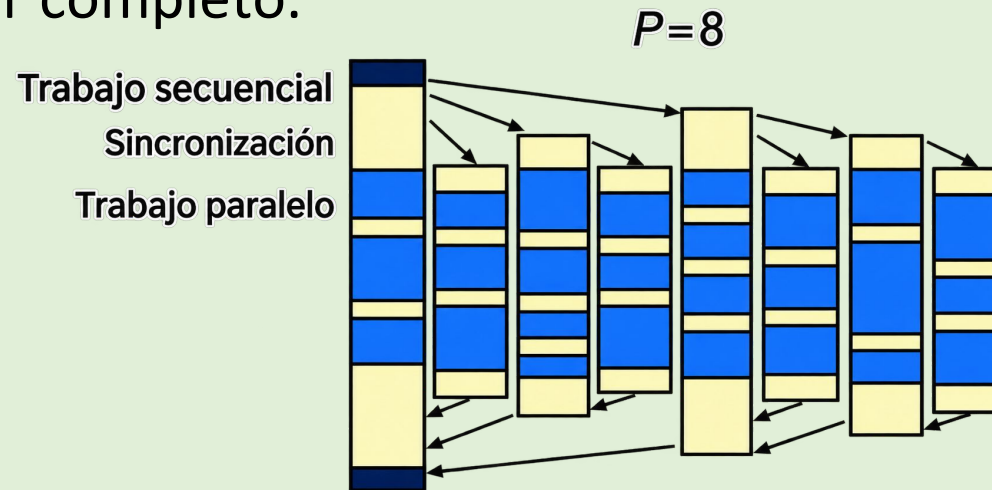
- La paralelización introduce una sobrecarga de trabajo adicional para iniciar y sincronizar las tareas.



- Las tareas adicionales debidas a la descomposición tienden a aumentar esta sobrecarga (overhead)

Sobrecarga (Overhead)

- El diseño cuidadoso de la sincronización puede reducir la sobrecarga, pero no puede eliminarla por completo.



- La gráfica representa una estructura de árbol para iniciar y sincronizar las tareas, de este modo el inicio es logarítmico en relación con el número de núcleos.

Las personas

- Detrás de cada línea de código hay al menos una persona.
- Aunque existen formas de pensar sobre estos problemas de manera estructurada, sigue siendo un arte.
- Al escribir el código se debe equilibrar la claridad con el rendimiento.
- También hay que considerar estas dificultades en la comunicación y luego escalarlas a equipos de proyectos

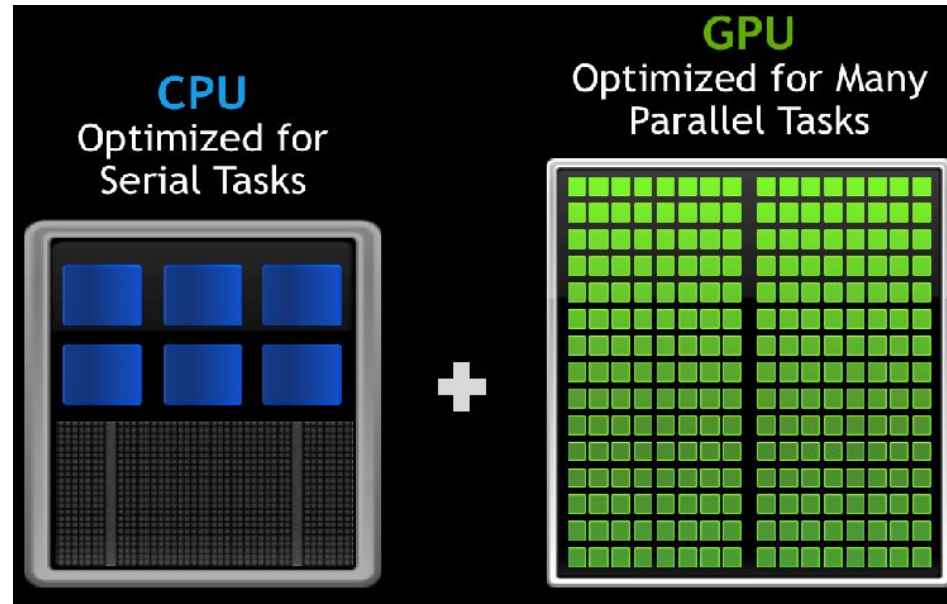


Contenido

- Introducción
 - ¿Qué es paralelismo a nivel de tarea?
- Grafo computacional
- Paralelismo ideal
- Planificación (scheduling) en multiprocesadores
- Análisis de algoritmos paralelos
 - Ley/Argumento de Amdahl
 - Ley de Gustafson-Barsis
- Retos y Dificultades en la computación paralela
- Computación heterogénea
- Referencias

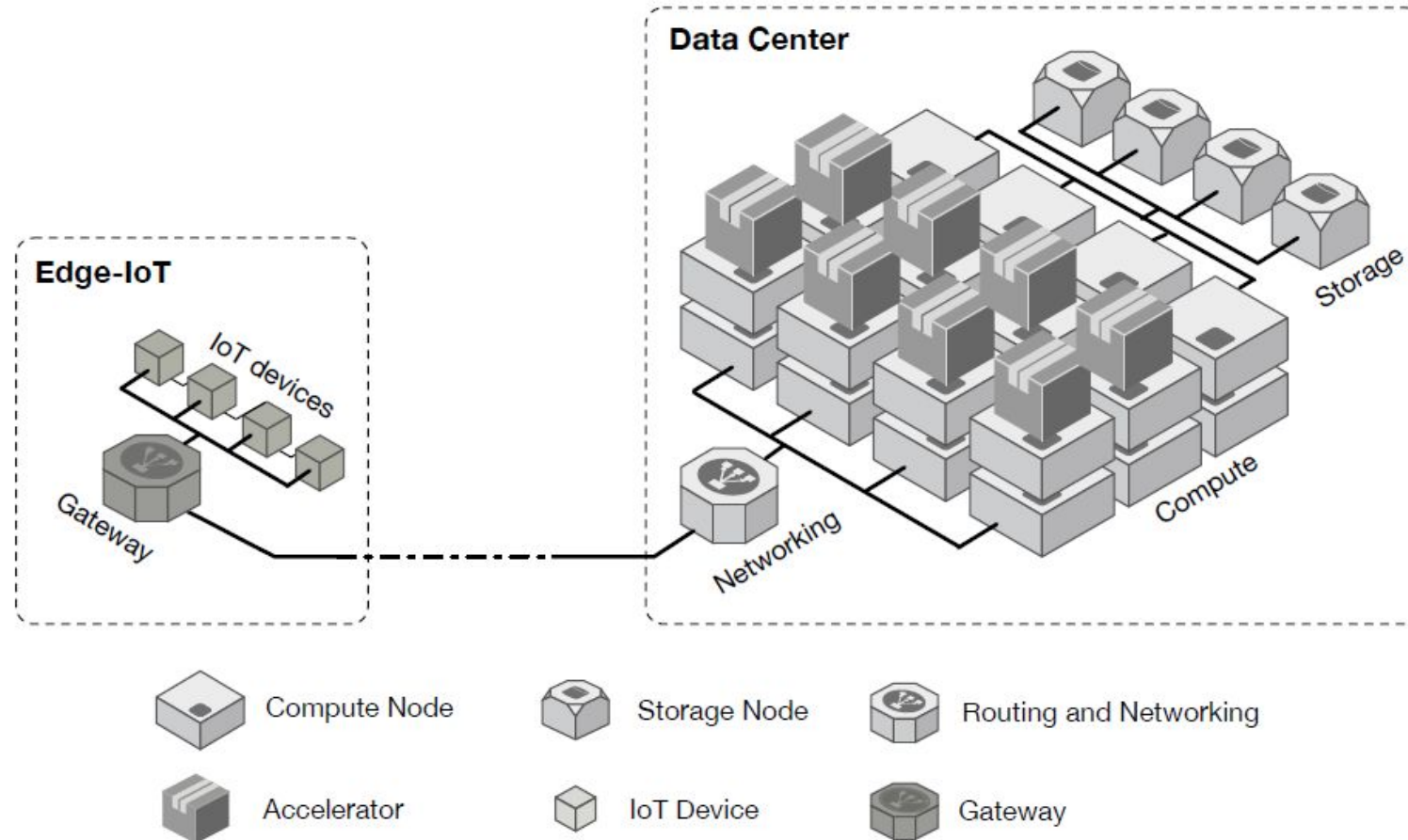
Computación heterogénea

Computación heterogénea



- Una implicación de la ley de Amdahl es que el mejoramiento en la rapidez (*speedup*) en aplicaciones reales que tienen porciones secuenciales y paralelas, requieren técnicas de computación heterogéneas. Por ejemplo, un procesador heterogéneo CPU-GPU puede proveer un desempeño más alto y ser más eficiente energéticamente que un procesador con solo CPU o con solo GPU.

Heterogeneidad desde el borde IoT hasta el Data Center

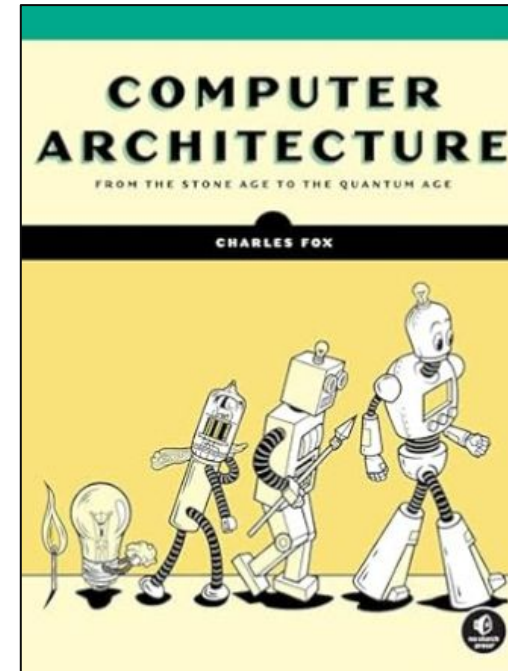


Heterogeneidad desde nivel de IoT (sensores inteligentes interconectados) al Data Center.

Arquitecturas_Paralelas

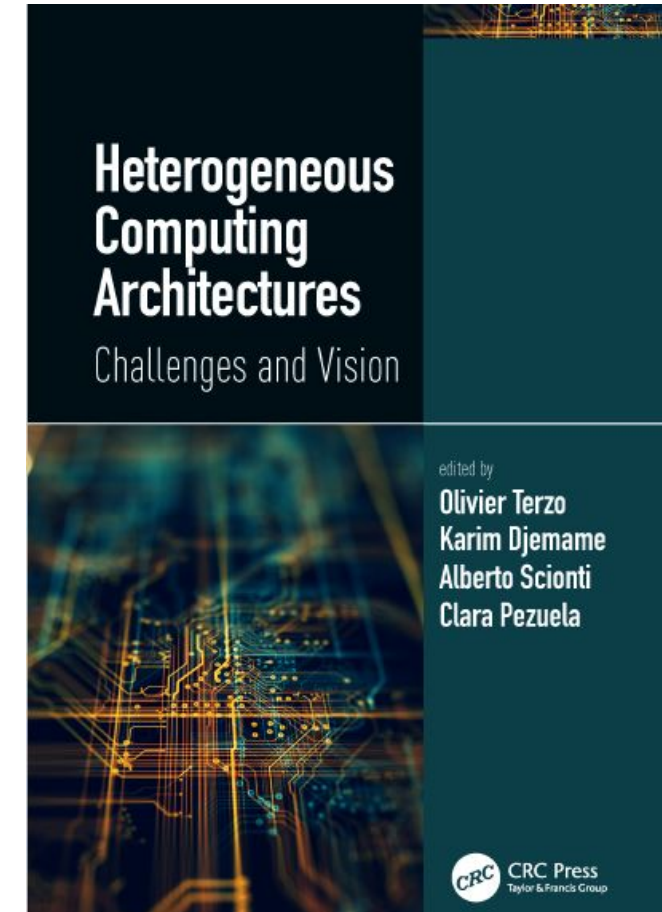
- La computación se está dividiendo en dos caminos:
 - los sistemas de bajo consumo, que forman el Internet de las cosas, y
 - los centros de computación de alto consumo, que forman la nube.
- En el capítulo 15 del libro se estudian arquitecturas de computadores paralelos como columna vertebral de la computación en la nube.

“Computer Architecture. From the Stone Age to the Quantum Age”. de **Fox, C.** No Starch Press. 2024



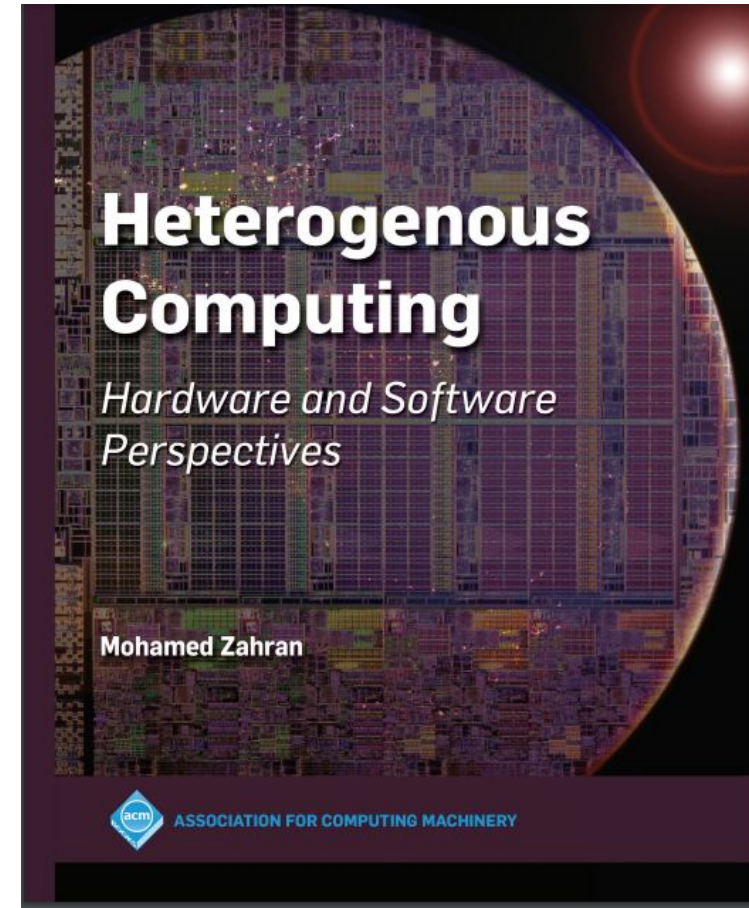
Heterogeneous Computing Architectures

- El libro mostrado tiene un enfoque interesante y actualizado sobre computación heterogénea.
- Mi propuesta de lectura inicial es
 - Revisar primero el capítulo 12 [Tiene como un resumen del libro] y
 - después leer los capítulos 1, 2 y 3



Heterogeneous Computing

- Why Are We Forced to Deal with Heterogeneous Computing?
- Different Players: Heterogeneity in Computing
- Architecture: Heterogeneity in Design
- Programmability
- Research Directions
 - Processing-in-Memory (PIM) / Near-Data Processing (NDP)
 - Exascale Computing
 - Neuromorphic Chips
 - Quantum Computing



Contenido

- Introducción
 - ¿Qué es paralelismo a nivel de tarea?
- Grafo computacional
- Paralelismo ideal
- Planificación (scheduling) en multiprocesadores
- Análisis de algoritmos paralelos
 - Ley/Argumento de Amdahl
 - Ley de Gustafson-Barsis
- Retos y Dificultades en la computación paralela
- Computación heterogénea
- Referencias

Referencias

- Blelloch, Guy. "Programming Parallel Algorithms". Communications of the ACM. 1996
- McCool, M.; Reinders, J.; Robison, A. *Structured Parallel Programming: Patterns for Efficient Computation*. Elsevier. 2013.
- Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford. Introduction to Algorithms (3rd ed.). MIT Press and McGraw-Hill. páginas 779–784. 2009.
- Gustafson, J. L. "Brent's Theorem". Encyclopedia of Parallel Computing. páginas 182–185. Springer . 2011
- Computer. "Amdahl's Law in the Multicore Era", 41 (7), 33–38, 2008

Gracias